

ВАСИЛЬКІВСЬКИЙ ФАХОВИЙ КОЛЕДЖ  
ДЕРЖАВНОГО НЕКОМЕРЦІЙНОГО ПІДПРИЄМСТВА  
«ДЕРЖАВНИЙ УНІВЕРСИТЕТ «КИЇВСЬКИЙ АВІАЦІЙНИЙ ІНСТИТУТ»  
(повне найменування вищого навчального закладу)

І ВІДДІЛЕННЯ ПІДГОТОВКИ ФАХІВЦІВ  
(повне найменування інституту, назва факультету (відділення))

ІНЖЕНЕРІЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ  
(повна назва кафедри (предметної, циклової комісії))

## **Пояснювальна записка**

до дипломного проекту (роботи)

"фаховий молодший бакалавр"

(освітньо-кваліфікаційний рівень)

На тему: Проектування та розробка інформаційної системи управління інфраструктурою  
інтернет-провайдера

Виконав: студент(ка) 4 курсу, групи 641  
напряму підготовки (спеціальності)  
121 «Інженерія програмного забезпечення»  
(шифр і назва напряму підготовки, спеціальності)

Тромса Д.С.  
(прізвище та ініціали)

Керівник: Галич Н.С.  
(прізвище та ініціали)

Рецензент: Романовська Л.О.  
(прізвище та ініціали)

## РЕЦЕНЗІЯ

на дипломну роботу студента(ки)

спеціальності 121 «Інженерія програмного забезпечення»

Тема роботи: **Проектування та розробка інформаційної системи управління інфраструктурою інтернет-провайдера**

Рецензент: \_\_\_\_\_ Романовська. Л.О \_\_\_\_\_  
(посада, місце роботи, прізвище та ініціали)

Дипломна робота виконана на тему розробки веборієнтованої інформаційної системи управління інфраструктурою інтернет-провайдера UIC Manager, реалізованої на Python 3.12 / Django 6.0.5 та розгорнутої за адресою <https://shtorm96.pythonanywhere.com/>. Робота містить три розділи, висновки, список літератури та додатки.

### Позитивні сторони роботи:

- чітка структура, логічна послідовність викладу та практична реалізація повнофункціональної системи (15 ORM-моделей, 17 URL-маршрутів, 2 management-команди);
- детальний порівняльний аналіз аналогів (WHMCS, SolarWinds, Zabbix, UTM5) та обґрунтований вибір технологій;
- реалізація інтерактивної мережевої карти (Leaflet.js / OpenStreetMap), білінгового модуля з обіцяними платежами та автоматичної генерації RADIUS-облікових даних;
- комплексне тестування: 76 перевірок за 6 категоріями, виявлено та усунено 7 дефектів;
- успішне виробниче розгортання на PythonAnywhere з відкритим кодом на GitHub

Зауваження: \_\_\_\_\_

---

Висновок: робота заслуговує оцінки

---

Рецензент: \_\_\_\_\_ Романовська Л.О. \_\_\_\_\_ «\_10\_» \_\_\_\_\_ Червень \_\_\_\_\_ 2026 р.

Інститут, факультет, відділення \_\_\_\_\_ І відділення  
Кафедра, циклова комісія \_\_\_\_\_ ІНЖЕНЕРІЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ  
Освітньо-кваліфікаційний рівень \_\_\_\_\_ ФАХОВИЙ МОЛОДШИЙ БАКАЛАВР  
Галузь знань \_\_\_\_\_ 12 «Інформаційні технології»  
(шифр і назва)  
Спеціальність \_\_\_\_\_ 121 «ІНЖЕНЕРІЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ»  
(шифр і назва)

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити): Вступ. РОЗДІЛ I. Аналіз предметної області та постановки задачі. РОЗДІЛ II. Розробка та реалізація системи. РОЗДІЛ III. Тестування та оцінка ефективності системи. Висновки. Список використаних джерел. Додатки

5. Перелік діаграм,скрінів,малюнків  
Робота має 6 малюнків та 4 діаграми.

---

6. Консультанти розділів проєкту (роботи)

| Розділ    | Прізвище, ініціали та посада консультанта | Підпис, дата   |                  |
|-----------|-------------------------------------------|----------------|------------------|
|           |                                           | завдання видав | завдання прийняв |
| II-III    | Галич Н.С.                                |                |                  |
| Рецензент | Романовська Л.О.                          |                |                  |

7. Дата видачі завдання: « 19 » березня 2026р.

**КАЛЕНДАРНИЙ ПЛАН**

| № з/п | Назва етапів дипломного проєкту (роботи) | Строк виконання етапів проєкту ( роботи ) | Дата та підпис студента | Підпис керівника ДП |
|-------|------------------------------------------|-------------------------------------------|-------------------------|---------------------|
| 1     | Збір інформації та вступне оформлення.   | 04.05 – 06.05.2026                        |                         |                     |
| 2     | Написання практичної частини.            | 07.05 – 12.05.2026                        |                         |                     |
| 3     | Розробка основної частини .              | 13.05 – 18.05.2026                        |                         |                     |
| 4     | Виконання спеціальної частини.           | 19.05 – 22.05.2026                        |                         |                     |
| 5     | Оформлення висновку.                     | 02.06 – 04.06.2026                        |                         |                     |
| 6     | Оформлення списку використаних джерел.   | 05.06 – 09.06.2026                        |                         |                     |
| 7     | Надання дипломного проєкту до захисту.   | 10.06.2026                                |                         |                     |
| 8     | Попередній захист                        | 18.06 – 19.06.2026                        |                         |                     |
|       |                                          |                                           |                         |                     |

**Студент**

\_\_\_\_\_  
(підпис)

Тромса Д.С.  
(прізвище та ініціали)

**Керівник проєкту (роботи)**

\_\_\_\_\_  
(підпис)

Галич Н.С.  
(прізвище та ініціали)

# Зміст

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>ВСТУП.....</b>                                              | <b>9</b>  |
| <b>РОЗДІЛ І. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ.....</b>                | <b>12</b> |
| 1.1. Загальна характеристика предметної області.....           | 12        |
| 1.2. Аналіз мережевих технологій інтернет-провайдерів.....     | 15        |
| 1.2.1. Технології доступу.....                                 | 15        |
| 1.2.2. Адресація та ідентифікація в мережі.....                | 16        |
| 1.2.3. Аутентифікація та авторизація абонентів.....            | 16        |
| 1.2.4. Специфіка обліку обладнання в ISP.....                  | 18        |
| 1.3. Аналіз існуючих рішень та аналогів.....                   | 19        |
| 1.4. Постановка задачі.....                                    | 20        |
| 1.5. Визначення вимог до програмного продукту.....             | 21        |
| 1.5.1. Функціональні вимоги.....                               | 21        |
| 1.5.2. Нефункціональні вимоги.....                             | 22        |
| <b>РОЗДІЛ ІІ. РОЗРОБКА ТА РЕАЛІЗАЦІЯ СИСТЕМИ.....</b>          | <b>24</b> |
| 2.1. Опис середовища розробки та обґрунтування технологій..... | 24        |
| 2.1.1. Мова програмування Python.....                          | 24        |
| 2.1.2. Веб-фреймворк Django 6.0.....                           | 24        |
| 2.1.3. База даних SQLite.....                                  | 25        |
| 2.1.4. Фронтенд: Bootstrap 5 та Leaflet.js.....                | 25        |
| 2.2. Проектування архітектури системи.....                     | 26        |
| 2.2.1. Загальна архітектура (MVC/MTV).....                     | 26        |
| 2.2.2. Структура проєкту.....                                  | 27        |
| 2.2.3. Маршрутизація URL.....                                  | 29        |
| 2.3. Моделювання та проектування бази даних.....               | 31        |
| 2.3.1. Ключові таблиці.....                                    | 31        |
| 2.3.2. Схема зв'язків між таблицями.....                       | 33        |
| 2.4. Опис алгоритмів роботи системи.....                       | 34        |
| 2.4.1. Алгоритм реєстрації нового клієнта.....                 | 34        |
| 2.4.2. Алгоритм місячного нарахування (monthly_charge).....    | 34        |
| 2.4.3. Алгоритм поповнення рахунку та розблокування.....       | 35        |
| 2.4.4. Алгоритм глобального пошуку.....                        | 35        |
| 2.4.5. Алгоритм реєстрації аварійної задачі.....               | 36        |
| 2.5. Реалізація програмного продукту.....                      | 36        |
| 2.5.1. Реалізація моделей (models.py).....                     | 36        |
| 2.5.2. Реалізація View-функцій.....                            | 37        |
| 2.5.3. Реалізація шаблону network_map.html.....                | 38        |
| 2.5.4. Management-команда monthly_charge.....                  | 38        |
| 2.6. Інтерфейс користувача.....                                | 39        |
| 2.6.1. Концепція дизайну.....                                  | 39        |
| 2.6.2. Опис екранів системи.....                               | 40        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 2.7. Розгортання системи .....                        | 41        |
| <b>РОЗДІЛ III. ТЕСТУВАННЯ ТА ВЕРИФІКАЦІЯ .....</b>    | <b>43</b> |
| 3.1. Методи тестування .....                          | 43        |
| 3.1.1. Модульне тестування (Unit Testing) .....       | 43        |
| 3.1.2. Функціональне та інтеграційне тестування ..... | 44        |
| 3.1.3. UI/UX тестування .....                         | 45        |
| 3.1.4. Навантажувальне тестування .....               | 45        |
| 3.1.5. Тестування безпеки .....                       | 45        |
| 3.2. Розробка тестових сценаріїв .....                | 46        |
| 3.3. Аналіз результатів тестування .....              | 48        |
| 3.4. Оцінка продуктивності системи .....              | 50        |
| 3.5. Порівняння з аналогами .....                     | 51        |
| 3.7. Рекомендації щодо впровадження .....             | 52        |
| 3.7.1. Порядок розгортання .....                      | 52        |
| 3.7.2. Першочергове наповнення даними .....           | 52        |
| 3.7.3. Перспективи розвитку .....                     | 53        |
| <b>ВИСНОВКИ .....</b>                                 | <b>54</b> |
| <b>СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ .....</b>               | <b>56</b> |
| <b>ДОДАТКИ .....</b>                                  | <b>57</b> |

## ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

| Скорочення  | Повне найменування                                            |
|-------------|---------------------------------------------------------------|
| ISP         | Internet Service Provider — інтернет-провайдер                |
| UIC Manager | Ukrainian ISP Controller Manager — розроблена система         |
| OLT         | Optical Line Terminal — оптичний лінійний термінал PON-мережі |
| ONU / ONT   | Optical Network Unit / Terminal — абонентський пристрій       |
| PON         | Passive Optical Network — пасивна оптична мережа              |
| GPON        | Gigabit PON — гігабітна PON-мережа (стандарт ITU-T G.984)     |
| RADIUS      | Remote Authentication Dial-In User Service — протокол AAA     |
| PPPoE       | Point-to-Point Protocol over Ethernet — протокол підключення  |
| MAC         | Media Access Control — фізична адреса мережевого пристрою     |
| ORM         | Object-Relational Mapping — об'єктно-реляційне відображення   |
| MTV/MVC     | Model-Template-View / Model-View-Controller — архіт. шаблон   |
| CSRF        | Cross-Site Request Forgery — міжсайтова підробка запитів      |
| XSS         | Cross-Site Scripting — міжсайтовий скриптинг (атака)          |
| СУБД        | Система управління базами даних                               |
| ДСТУ        | Державний стандарт України                                    |

|      |                                                             |
|------|-------------------------------------------------------------|
| UML  | Unified Modeling Language — уніфікована мова моделювання    |
| API  | Application Programming Interface — інтерфейс програмування |
| WSGI | Web Server Gateway Interface — шлюзовий інтерфейс Python    |



## ВСТУП

Інтернет-провайдери щодня виконують велику кількість операцій, пов'язаних з обслуговуванням абонентів і підтримкою мережі. Це підключення нових клієнтів, облік платежів, зміна тарифів, реєстрація звернень і контроль роботи обладнання. Поки абонентів небагато, такі процеси часто ведуть у звичайних таблицях або окремих журналах. Але коли база зростає, цей спосіб стає незручним і призводить до втрати даних, дублювання інформації та зайвих витрат часу на щоденну роботу.

Під час аналізу роботи регіональних провайдерів я побачив, що дані про абонентів, обладнання, платежі та аварії часто зберігаються в різних місцях — окремих файлах чи програмах. Через це важко швидко отримати потрібну інформацію, а працівники витрачають більше часу на пошук, ніж на саму роботу з клієнтами.

Готові програми для провайдерів не завжди підходять невеликим компаніям. Одні коштують дорого, інші вимагають довгого налаштування або містять багато зайвих функцій. Тому виникає потреба у простій веборієнтованій системі, яка б поєднувала облік абонентів, обладнання, білінг та аварійні роботи в одному місці.

Саме для цього розроблено систему UIC Manager. Вона призначена для автоматизації основних робочих процесів регіонального інтернет-провайдера: зберігає всі дані в одному місці та дає доступ до них і операторам, і технічним працівникам через веб-інтерфейс.

**Метою кваліфікаційної роботи є проектування та розробка веборієнтованої інформаційної системи управління інфраструктурою інтернет-провайдера — UIC Manager (Ukrainian ISP Controller Manager), яка забезпечує комплексну автоматизацію процесів управління мережевим обладнанням, абонентською базою, білінгом, персоналом, складом матеріально-технічних ресурсів та аварійними ситуаціями.**

|             |  |             |        |      |               |  |  |                   |      |        |  |
|-------------|--|-------------|--------|------|---------------|--|--|-------------------|------|--------|--|
|             |  |             |        |      | 641.23.26. ДП |  |  |                   |      |        |  |
| Зм.         |  | № докум.    | Підпис | Дата |               |  |  |                   |      |        |  |
| Розроб.     |  | Тромса Д.С. |        |      | Вступ         |  |  | Літ.              | Лист | Листів |  |
| Перевір.    |  | Галич Н.С.  |        |      |               |  |  |                   | 9    | 66     |  |
| Консультант |  |             |        |      |               |  |  | ВФК ДНП «ДУ «КАІ» |      |        |  |
| Н. Контр.   |  |             |        |      |               |  |  |                   |      |        |  |
| Затверд.    |  |             |        |      |               |  |  |                   |      |        |  |

Для досягнення поставленої мети в роботі вирішуються такі задачі:

- аналіз предметної області та специфіки роботи інтернет-провайдерів;
- дослідження мережевих технологій та стандартів, що застосовуються в ISP;
- дослідження існуючих програмних рішень для управління ISP-інфраструктурою;
- визначення функціональних та нефункціональних вимог до системи;
- обґрунтування вибору технологічного стеку для реалізації системи;
- проектування архітектури програмної системи за шаблоном MVC;
- розробка реляційної моделі бази даних;
- реалізація модулів системи: обладнання, клієнти, білінг, тікети, склад, персонал;
- розробка інтуїтивного веб-інтерфейсу з адаптивним дизайном;
- тестування системи та аналіз результатів;
- оцінка продуктивності та економічне обґрунтування впровадження.

Об'єктом дослідження є процеси управління мережевою інфраструктурою та абонентською базою інтернет-провайдера.

Предметом дослідження є інформаційна система UIC Manager, розроблена для автоматизації операційної діяльності регіонального інтернет-провайдера.

**Методи дослідження.** Для вивчення теми я проаналізував роботу реальних провайдерів і кілька наявних програм-аналогів. Систему розробляв поступово: спочатку базовий облік клієнтів і нарахування плати, потім додав карту, заявки та склад. Кожен модуль перевіряв вручну, а швидкодію — на тестових базах від 100 до 3000 записів.

**Практичне значення роботи** полягає у тому, що розроблена система може бути безпосередньо впроваджена в операційну діяльність малих і середніх інтернет-провайдерів в Україні. Зокрема, прототип системи розроблявся з урахування реальних бізнес-процесів регіонального провайдера міста Васильків Київської області, що підтверджує прикладну цінність розробки.

Структура роботи. Кваліфікаційна робота складається зі вступу, трьох розділів, висновків, списку використаних джерел (20 позицій). Загальний обсяг роботи — 66 сторінки, 6 рисунків, 16 таблиць.

## РОЗДІЛ І. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

### 1.1. Загальна характеристика предметної області

Інтернет-провайдер (ISP, Internet Service Provider) — це компанія, яка надає доступ до Інтернету людям і організаціям. На практиці провайдер одночасно займається трьома різними напрямками роботи: технічним (мережа і обладнання), комерційним (клієнти і тарифи) та фінансовим (платежі і облік). Щоб усе це працювало злагоджено, потрібно координувати багато процесів одночасно.

За даними НКРЗІ, в Україні працює понад 1200 провайдерів. Серед них є великі магістральні оператори, які забезпечують транзит трафіку між регіонами, і малі регіональні компанії, що обслуговують один-два райони міста. Саме малі провайдери становлять більшість і саме їм найбільше бракує доступних інструментів автоматизації — на дорогі корпоративні системи в них немає бюджету.

Діяльність інтернет-провайдера структурно поділяється на кілька ключових підрозділів:

- Технічний відділ — відповідає за будівництво та обслуговування мережевої інфраструктури (прокладання кабелів, монтаж обладнання), підключення нових абонентів, усунення аварій та технічних несправностей, планове технічне обслуговування вузлів доступу.
- Абонентський відділ (call-center, операторський відділ) — прийом звернень від клієнтів по телефону та онлайн, реєстрація заявок на підключення та технічних скарг, консультація абонентів щодо послуг та тарифів, оформлення договорів.

|             |  |            |        |      |                   |      |        |
|-------------|--|------------|--------|------|-------------------|------|--------|
|             |  |            |        |      | 641.23.26. ДП     |      |        |
|             |  |            |        |      |                   |      |        |
| Зм.         |  | № докум.   | Підпис | Дата | Розділ 1          |      |        |
| Розроб.     |  | Тромса Д.С |        |      |                   |      |        |
| Перевір.    |  | Галич Н.С. |        |      |                   |      |        |
| Консультант |  |            |        |      |                   |      |        |
| Н. Контр.   |  |            |        |      |                   |      |        |
| Затверд.    |  |            |        |      |                   |      |        |
|             |  |            |        |      | Літ.              | Лист | Листів |
|             |  |            |        |      |                   | 12   | 66     |
|             |  |            |        |      | ВФК ДНП «ДУ «КАІ» |      |        |

- Фінансовий відділ (бухгалтерія) — облік надходжень від абонентів, нарахування абонентської плати, контроль дебіторської заборгованості, формування фінансової звітності.
- Склад та матеріально-технічне забезпечення — облік і видача мережевого обладнання та витратних матеріалів для монтажу і ремонту (кабелі, роз'єми, патч-корди, оптичні муфти, роутери для клієнтів).

Мережева інфраструктура сучасного регіонального провайдера включає такі ключові компоненти:

- Активне мережеве обладнання: маршрутизатори (роутери) — для міжмережевої маршрутизації пакетів; комутатори (свічі) — для комутації трафіку всередині вузлів; OLT-термінали (Optical Line Terminal) — центральне обладнання PON-мереж, що обслуговує десятки портів підключення абонентів;
- Пасивна мережева інфраструктура: оптоволоконний кабель різних типів (одномодовий G.652D, G.657A для PON), мідний кабель (Ethernet Cat5e/Cat6), оптичні муфти для з'єднання кабелів, розподільні шафи та боксах на вузлах доступу, оптичні спліттери для розгалуження PON-сигналу;
- Серверна інфраструктура: DHCP-сервери для динамічної видачі IP-адрес, RADIUS-сервери для аутентифікації та авторизації абонентів, сервери моніторингу мережі, резервне копіювання;
- Термінальне обладнання абонентів: ONT/ONU-пристрої (Optical Network Terminal/Unit) — підключаються до оптичного кабелю в квартирі абонента та видають Ethernet або Wi-Fi; домашні роутери абонентів; медіаконвертери для переходу з оптики на мідь;
- Системи білінгу та обліку: програмне забезпечення для нарахування абонентської плати, контролю балансу та формування звітів.

Важливим аспектом є географічний розподіл мережевої інфраструктури. Обладнання встановлюється на вузлах доступу — розподільних шафах та боксах, що розміщуються у різних районах міста чи населеного пункту (у підвалах будинків,

на опорах, у вуличних шафах). Відповідно, технічним фахівцям необхідно мати чіткий реєстр місць розташування обладнання з адресами та координатами — для відображення на карті та оперативного виїзду у разі аварії.

Тарифна політика провайдерів зазвичай включає кілька пакетів послуг, що відрізняються швидкістю доступу до Інтернету, наявністю телебачення (IPTV), типом IP-адресації (статична або динамічна). Часто пропонуються акційні тарифи для нових абонентів та спеціальні пакети для бізнес-клієнтів. Система управління повинна підтримувати зміну тарифного плану та автоматичне нарахування відповідної абонентської плати.

Аварійні ситуації є невід'ємною частиною роботи провайдера. Вони поділяються на заплановані (профілактичне технічне обслуговування) та позапланові (обрив кабелю, вихід з ладу обладнання, повінь у підвалі з обладнанням). У разі аварії необхідно швидко зафіксувати проблему, призначити відповідальних фахівців, відстежити хід усунення та повідомити абонентів. Ефективна система управління аварійними задачами суттєво скорочує MTTR (Mean Time To Restore — середній час відновлення), що є критично важливим для підтримання рівня якості послуг.

Ключові бізнес-процеси ISP можна класифікувати за трьома вимірами:

Технічне управління інфраструктурою охоплює облік мережевого обладнання з визначенням географічного розташування, моніторинг стану вузлів (онлайн/офлайн), реєстрацію та ескалацію аварій, управління бригадами технічних фахівців, планування будівництва нових ділянок мережі.

Комерційне управління включає ведення реєстру абонентів з повними контактними даними, управління договорами та тарифними планами, обробку заявок на нові підключення, розгляд претензій та скарг, інформування клієнтів про аварії та профілактичні роботи.

Фінансовий облік та білінг охоплює ведення особових рахунків абонентів, нарахування щомісячної абонентської плати, облік платежів (готівка, безготівковий розрахунок, платіжні системи), контроль заборгованості, видачу обіцяних платежів, формування фінансової звітності.

Коли всі ці процеси ведуться вручну в різних таблицях, виникають типові проблеми: дані дублюються і суперечать одне одному, у нарахуваннях трапляються помилки, а на пошук інформації йде багато часу. Єдина система обліку дозволяє цього уникнути.

## **1.2. Аналіз мережевих технологій інтернет-провайдерів**

Перш ніж проєктувати систему, я розібрався, як саме провайдер підключає абонентів і які дані про кожен пристрій потрібно зберігати. Це безпосередньо вплинуло на те, які поля з'явилися в базі даних UIC Manager.

### **1.2.1. Технології доступу**

Сучасні регіональні провайдери використовують переважно два типи технологій для підключення абонентів: FTTH (Fiber to the Home) на базі технологій PON та Ethernet. Застарілі технології DSL (ADSL, VDSL) поступово витісняються оптоволоконними рішеннями завдяки різкому здешевленню оптичного обладнання.

PON (Passive Optical Network) — пасивна оптична мережа — є домінуючою технологією для нових розгортань. В архітектурі PON між центральним обладнанням (OLT) та абонентськими пристроями (ONU/ONT) використовуються лише пасивні оптичні спліттери (розгалужувачі), що не потребують електроживлення. Це суттєво знижує операційні витрати та підвищує надійність. Кожен абонент ідентифікується унікальним серійним номером ONU-пристрою (serial number, або LOID/SLID), що використовується для реєстрації в системі управління OLT. Цей номер є критично важливою даниною, що повинна зберігатися в системі управління провайдера.

Основні стандарти PON:

- GPON (Gigabit PON, ITU-T G.984) — найбільш поширений стандарт у регіональних мережах, забезпечує до 2,5 Гбіт/с вниз і 1,25 Гбіт/с вгору, коефіцієнт спліттування до 1:128; типова відстань від OLT до ONT — до 20 км;
- XGS-PON (10-Gigabit Symmetric PON, ITU-T G.9807.1) — сучасний стандарт, симетричний 10 Гбіт/с, активно впроваджується у нових мережах;

- EPON/GEPON (Ethernet PON, IEEE 802.3ah) — використовує Ethernet-кадри, поширений в Азії та частково в Україні у ранніх розгортаннях.

Ethernet-доступ на базі Passive Ethernet (FTTx) застосовується у щільно забудованих районах. Кожен будинковий вузол обладнується керованим комутатором (managed switch), до якого підключаються квартирні роутери абонентів через патч-корди або вертикальний кабель. Кожен порт комутатора відповідає одному абоненту, що дозволяє адміністратору відстежувати статус підключення.

### **1.2.2. Адресація та ідентифікація в мережі**

Управління IP-адресацією є важливим завданням для ISP. Провайдер отримує від регіонального реєстру (RIPE NCC для Європи та України) пул публічних IP-адрес, які розподіляються між абонентами та мережевою інфраструктурою.

Можливі два варіанти призначення IP-адрес абонентам:

- Статична IP-адреса — фіксована адреса, що постійно призначається конкретному абоненту незалежно від часу підключення. Зазвичай пропонується як додаткова опція в преміум-тарифах. Статична IP необхідна для серверів, відеоспостереження та VPN.
- Динамічна IP-адреса — адреса видається з пулу при кожному підключенні через DHCP-сервер і прив'язується до MAC-адреси пристрою або серійного номера ONU. При відключенні адреса повертається у пул і може бути видана іншому абоненту.

MAC-адреса (Media Access Control) — унікальний ідентифікатор мережевого пристрою на канальному рівні моделі OSI. Формат: шість октетів у шістнадцятковому записі, розділені двокрапками або дефісами: 00:1A:2B:3C:4D:5E. Перші три октети (OUI — Organizationally Unique Identifier) ідентифікують виробника пристрою. MAC-адреса обладнання провайдера є важливою даниною для ідентифікації конкретного вузла мережі при технічному обслуговуванні.

### **1.2.3. Аутентифікація та авторизація абонентів**

Для аутентифікації абонентів при підключенні до мережі використовуються такі протоколи та механізми:



- PPPoE (Point-to-Point Protocol over Ethernet) — широко застосовується для аутентифікації через логін та пароль. Абонент налаштовує PPPoE-клієнт на своєму роутері, вводить логін та пароль, після чого відбувається аутентифікація на RADIUS-сервері провайдера. Перевага: зручне управління сесіями, облік трафіку; недолік: додаткове навантаження на обладнання.
- IPoE (IP over Ethernet) — більш сучасний підхід, де ідентифікація виконується за MAC-адресою або серійним номером ONU без введення логіну/паролю. Абонент отримує IP-адресу через DHCP автоматично. Перевага: простота для абонента; недолік: складніше управління сесіями.
- DHCP Option 82 (Relay Agent Information) — механізм вставки додаткової інформації про порт комутатора або OLT у DHCP-запит. Дозволяє DHCP-серверу ідентифікувати абонента за фізичним портом підключення без MAC-адреси.

RADIUS (Remote Authentication Dial-In User Service) — протокол для централізованої аутентифікації, авторизації та обліку (AAA — Authentication, Authorization, Accounting). RADIUS-сервер зберігає базу даних облікових записів абонентів та управляє доступом до мережі. Пара «логін + пароль», що генерується системою UIC Manager при реєстрації клієнта, може безпосередньо використовуватися для конфігурації RADIUS-сервера.

Усі ці технічні деталі прямо визначили, які поля потрібні в системі: IP- і MAC-адреса обладнання, серійний номер ONU абонента, тариф зі швидкістю, а також логін і пароль для RADIUS. Усе це реалізовано в моделях UIC Manager.

| Технологія | Стандарт       | Ідентифікатор абонента | Швидкість       | Поширеність в Україні |
|------------|----------------|------------------------|-----------------|-----------------------|
| GPON       | ITU-T G.984    | Serial Number ONU      | 2.5/1.25 Гбіт/с | Висока                |
| XGS-PON    | ITU-T G.9807.1 | Serial Number ONU      | 10/10 Гбіт/с    | Зростаюча             |
| EPON       | IEEE 802.3ah   | MAC ONU                | 1/1 Гбіт/с      | Середня               |

|                  |                  |                        |                    |            |
|------------------|------------------|------------------------|--------------------|------------|
| Ethernet<br>FTTx | IEEE 802.3       | MAC порт<br>комутатора | 100–1000<br>Мбіт/с | Висока     |
| DSL<br>(VDSL2)   | ITU-T<br>G.993.2 | Лінійний порт<br>DSLAM | 100/50<br>Мбіт/с   | Знижується |

*Таблиця 1.1 — Порівняння технологій доступу для ISP*

#### 1.2.4. Специфіка обліку обладнання в ISP

Окремої уваги заслуговує питання обліку мережевого обладнання в умовах регіонального провайдера. На відміну від корпоративних мереж, де обладнання концентрується у серверних кімнатах, мережа ISP є географічно розподіленою. Типова регіональна мережа провайдера міста з населенням 40 000–100 000 мешканців може включати:

- 2–5 вузлів рівня ядра (core layer): потужні маршрутизатори рівня підприємства, з'єднані оптичними лінками, часто розташовані у власних або орендованих серверних приміщеннях;
- 10–50 вузлів рівня агрегації (aggregation layer): комутатори агрегації, що з'єднують вузли доступу, розташовані у розподільних шафах мікрорайонів;
- 50–300 вузлів рівня доступу (access layer): OLT-модулі або комутатори доступу, встановлені безпосередньо у будинках або вуличних шафах;
- 500–5000 абонентських ONU/ONT-пристроїв, встановлених у квартирах та офісах.

Для ефективного управління такою розподіленою інфраструктурою необхідно мати єдиний реєстр обладнання зі зрозумілими атрибутами. Ключові атрибути записів обладнання:

Назва — людиночитаний ідентифікатор, що відображає тип та місце розташування пристрою. Рекомендована конвенція найменування: [місто]-[район]-[тип]-[номер], наприклад: Vas-Center-OLT-01, Vas-North-SW-003. Чітка конвенція найменування суттєво спрощує пошук та ідентифікацію обладнання.

IP-адреса є унікальним мережевим ідентифікатором пристрою. У мережах

провайдерів адміністративні IP-адреси обладнання зазвичай знаходяться у приватних діапазонах (наприклад, 10.0.0.0/8) або окремій підмережі управління (Out-of-Band Management Network). Унікальність IP-адреси в межах системи обліку є обов'язковою.

Географічні координати (широта та довгота) дозволяють відобразити обладнання на інтерактивній карті та автоматично прокласти маршрут для технічного фахівця при виїзді на аварію. Координати можна визначити через Google Maps, OpenStreetMap або GPS-навігатор при виїзді на монтаж.

Статус активності (online/offline) є ключовим оперативним показником. В ідеальній системі цей статус оновлюється автоматично через ICMP-опитування або SNMP. У розробленій системі UIC Manager статус встановлюється вручну оператором або технічним фахівцем після перевірки — це обмеження, що планується усунути шляхом інтеграції з SNMP-опитувальником у майбутній версії.

### **1.3. Аналіз існуючих рішень та аналогів**

На ринку програмного забезпечення для управління ISP-бізнесом існує ряд рішень різного класу. Розглянемо основні категорії та конкретні продукти, оцінимо їх відповідність вимогам малого регіонального провайдера.

WHMCS — популярна комерційна платформа для хостинг-провайдерів. Має сильний модуль білінгу і клієнтів, але повністю орієнтована на хостинг: у ній немає обліку фізичного мережевого обладнання, карти вузлів, аварійних задач та складу. Інтерфейс англomовний, ліцензія платна (від 20 USD на місяць).

SolarWinds — потужна система моніторингу мережі (модулі NPM, NCM, IPAM). Добре показує стан обладнання, але не має ні білінгу, ні обліку клієнтів, ні роботи із заявками. Вартість від 1500 USD на рік робить її недоступною для малого провайдера.

Zabbix — безкоштовна система моніторингу з відкритим кодом. Чудово стежить за обладнанням, але це суто технічний інструмент: клієнтів, білінгу і тікетів у ній немає взагалі. Налаштування потребує серйозних технічних знань.

UTM5 — білінгова система, поширена серед провайдерів пострадянського

простору. Має білінг і підтримку RADIUS, але архітектурно застаріла (десктопний клієнт замість веб-інтерфейсу), не має карти обладнання та аварійних задач.

Підсумовуючи: кожна з систем закриває лише частину потреб. Технічні рішення (Zabbix, SolarWinds) не мають білінгу, а білінгові (UTM5, WHMCS) — обліку обладнання з картою. Жодне з них не є україномовним і безкоштовним водночас. Саме цю прогалину закриває UIC Manager, об'єднуючи всі ці функції в одній системі.

#### **1.4. Постановка задачі**

На основі аналізу предметної області та існуючих рішень сформульовано задачу кваліфікаційної роботи.

Необхідно розробити веборієнтовану інформаційну систему управління інфраструктурою інтернет-провайдера UIC Manager, що забезпечує:

1. Реєстрацію та облік мережевого обладнання: назва, IP-адреса, MAC-адреса, розташування, геокоординати, статус онлайн/офлайн.
2. Управління клієнтською базою: реєстрація абонентів, персональні дані, адреса, телефон, серійний номер ONU, прив'язка до обладнання та тарифного плану.
3. Повнофункціональний білінговий модуль: рахунки абонентів, поповнення, списання, обіцяні платежі, блокування за несплату.
4. Систему тікетів та заявок: реєстрація звернень, заявки на підключення, призначення виконавців, відстеження статусів.
5. Управління аварійними задачами з геолокацією та призначенням бригад.
6. Складський облік матеріально-технічних ресурсів та торгівлю мережевим обладнанням.
7. Управління персоналом: реєстрація співробітників, посад та контактних даних.
8. Систему повідомлень та масового розсилання клієнтам.
9. Управління тарифними планами з описом умов та цін.
10. Аутентифікацію та авторизацію адміністраторів системи.

Система повинна мати зручний веб-інтерфейс, що дозволяє операторам і

технічним фахівцям ефективно виконувати свої функції без спеціальної технічної підготовки. Архітектурно система реалізується як веб-застосунок з клієнт-серверною архітектурою: серверна частина забезпечує обробку бізнес-логіки та доступ до бази даних, клієнтська частина (браузер) — відображення інтерфейсу.

Система повинна підтримувати одночасну роботу кількох операторів і зберігати повну аудиторну стежку дій (журнали платежів, журнали повідомлень). Всі дані повинні зберігатися в реляційній базі даних з підтримкою посилальної цілісності.

## **1.5. Визначення вимог до програмного продукту**

### **1.5.1. Функціональні вимоги**

Модуль управління обладнанням (FV-1):

- FV-1.1: система дозволяє додавати, редагувати та видаляти записи про мережеве обладнання;
- FV-1.2: для кожного обладнання зберігається: назва, IP-адреса (IPv4/IPv6 з валідацією), MAC-адреса, текстовий опис розташування, географічні координати, статус активності;
- FV-1.3: система відображає мережеву карту OpenStreetMap з маркерами місць розташування обладнання з кольоровим кодуванням за статусом;
- FV-1.4: реалізовано фільтрацію та пошук обладнання за ключовими полями (назва, IP, MAC, розташування).

Модуль управління клієнтами (FV-2):

- FV-2.1: реєстрація нових абонентів із збереженням ПІБ, номера договору (унікальний), адреси, телефону, серійного номера ONU (унікальний);
- FV-2.2: при реєстрації клієнта автоматично генеруються унікальний логін (7 цифр) та пароль (буква+4 цифри) для RADIUS-аутентифікації;
- FV-2.3: прив'язка клієнта до тарифного плану та вузла мережевого обладнання;
- FV-2.4: ручне та автоматичне блокування/розблокування клієнтів;
- FV-2.5: відображення статусу підключення (онлайн/офлайн) клієнта.

Модуль білінгу (FV-3):

- FV-3.1: ведення особових рахунків абонентів з поточним балансом;
- FV-3.2: операції поповнення рахунку та ручного списання з обов'язковим коментарем;
- FV-3.3: обіцяні платежі з датою закінчення та автоматичним зняттям блокування;
- FV-3.4: повний журнал всіх фінансових операцій (тип, сума, дата, виконавець);
- FV-3.5: автоматичне місячне нарахування абонентської плати через management-команду.

Модуль тікетів та заявок (FV-4):

- FV-4.1: реєстрація звернень клієнтів (тікетів) з описом, пріоритетом та статусом;
- FV-4.2: заявки на підключення нових абонентів з адресою, типом підключення та запланованою датою;
- FV-4.3: призначення відповідальних виконавців на задачі та заявки;
- FV-4.4: управління статусами задач: нова (new), в роботі (in\_progress), виконана (done).

### 1.5.2. Нефункціональні вимоги

| ID    | Категорія       | Вимога                             | Цільова метрика                          |
|-------|-----------------|------------------------------------|------------------------------------------|
| НФВ-1 | Продуктивність  | Час відповіді на типовий GET-запит | $\leq 2000$ мс                           |
| НФВ-2 | Надійність      | Доступність системи                | $\geq 99\%$ ( $\leq 87$ год/рік простою) |
| НФВ-3 | Безпека         | Захист від веб-атак                | CSRF, XSS, SQLi захист (Django)          |
| НФВ-  | Масштабованість | Максимальна                        | До 5000 без деградації                   |

|        |                   |                               |                                          |
|--------|-------------------|-------------------------------|------------------------------------------|
| 4      |                   | кількість абонентів           |                                          |
| НФВ-5  | Зручність         | Час освоєння новим оператором | ≤ 2 години без навчання                  |
| НФВ-6  | Локалізація       | Мова інтерфейсу               | Українська мова                          |
| НФВ-7  | Кросбраузерність  | Підтримка браузерів           | Chrome, Firefox, Edge (останні 2 версії) |
| НФВ-8  | Адаптивність      | Мінімальна ширина екрана      | від 360px (смартфони)                    |
| НФВ-9  | Супроводжуваність | Документація                  | Технічна та користувацька документація   |
| НФВ-10 | Переносимість     | ОС серверного середовища      | Linux Ubuntu 20.04 LTS+                  |

*Таблиця 1.3 — Нефункціональні вимоги до системи*

В системі виділено три ролі користувачів: Адміністратор (повний доступ до всіх функцій, включно з управлінням конфігурацією та звітністю), Оператор (доступ до функцій роботи з клієнтами, білінгом та тікетами), Технічний фахівець (доступ до задач та карти обладнання). Аутентифікація реалізована через стандартний механізм Django Authentication.

## РОЗДІЛ II. РОЗРОБКА ТА РЕАЛІЗАЦІЯ СИСТЕМИ

### 2.1. Опис середовища розробки та обґрунтування технологій

#### 2.1.1. Мова програмування Python

Основною мовою я обрав Python 3.12. Головна причина проста: я вже маю досвід роботи з цією мовою, а разом із Django вона дозволяє швидко зробити робочий веб-застосунок без зайвого коду. Python також зручний для роботи з рядками — це знадобилось при автоматичній генерації логінів і паролів для кожного нового клієнта.

- висока читабельність коду завдяки обов'язковому відступу (indentation-based syntax) та чіткому синтаксису, що полегшує подальшу підтримку;
- висока швидкість розробки: Python-код зазвичай вдвічі коротший за еквівалентний Java-код;
- розвинена екосистема бібліотек (PyPI налічує більше 500 тисяч пакетів);
- нативна підтримка Unicode на рівні мови, що критично важливо для українськомовного інтерфейсу;
- широке застосування у веброзробці: Django, FastAPI, Flask;
- відмінна підтримка роботи з реляційними БД через ORM та драйвери.

#### 2.1.2. Веб-фреймворк Django 6.0

Django 6.0.5 обрано через його вбудований ORM — він дозволяє описати всі таблиці бази даних звичайними Python-класами і автоматично отримати захист від SQL-ін'єкцій, систему міграцій та готову адмінпанель. Завдяки цьому я зосередився на логіці системи, а не на написанні SQL вручну. Django працює за шаблоном MTV (Model-Template-View): модель відповідає за дані, шаблон — за вигляд сторінки, view — за обробку запитів.

|             |  |             |        |      |               |  |  |                   |      |        |    |
|-------------|--|-------------|--------|------|---------------|--|--|-------------------|------|--------|----|
|             |  |             |        |      | 641.23.26. ДП |  |  |                   |      |        |    |
|             |  |             |        |      |               |  |  |                   |      |        |    |
| Зм.         |  | № докум.    | Підпис | Дата |               |  |  |                   |      |        |    |
| Розроб.     |  | Тромса Д.С. |        |      | Розділ 2      |  |  | Літ.              | Лист | Листів |    |
| Перевір.    |  | Галич Н.С.  |        |      |               |  |  |                   |      | 24     | 66 |
| Консультант |  |             |        |      |               |  |  | ВФК ДНП «ДУ «КАІ» |      |        |    |
| Н. Контр.   |  |             |        |      |               |  |  |                   |      |        |    |
| Затверд.    |  |             |        |      |               |  |  |                   |      |        |    |



- ORM для абстракції роботи з базами даних без написання SQL;
- система міграцій для версіонування схеми БД (python manage.py migrate);
- потужний адміністративний інтерфейс Django Admin;
- вбудована система аутентифікації та авторизації;
- автоматичний захист від CSRF, XSS, SQL-ін'єкцій;
- шаблонізатор Django Template Language (DTL);
- гнучка система маршрутизації URL;
- система форм та валідації даних.

### 2.1.3. База даних SQLite

Базою даних обрано SQLite. Її головна перевага для цього проєкту — вся база зберігається в одному файлі db.sqlite3, без окремого сервера СУБД. Це сильно спрощує і резервне копіювання, і розгортання на PythonAnywhere. При тестуванні з базою у 3000 клієнтів швидкодії вистачало з запасом, а якщо база виросте — Django дозволяє перейти на PostgreSQL зміною кількох рядків у налаштуваннях.

### 2.1.4. Фронтенд: Bootstrap 5 та Leaflet.js

Для зовнішнього вигляду я використав Bootstrap 5 — він дав готову адаптивну верстку і набір компонентів (таблиці, форми, картки) без написання власного CSS. Для карти обрано Leaflet.js — невелику бібліотеку, що працює з безкоштовними картами OpenStreetMap. Саме ця пара дозволила зробити інтерактивну карту мережі без платних картографічних сервісів.

Leaflet.js — легковажна (≈44 КБ) open-source бібліотека для інтерактивних веб-карт. Підтримує тайлові шари (OpenStreetMap, Mapbox), маркери, події. Використовується безкоштовно з тайлами OpenStreetMap без ліцензійних обмежень.

| Технологія | Версія | Призначення        | Ліцензія       |
|------------|--------|--------------------|----------------|
| Python     | 3.12.x | Мова програмування | PSF (відкрита) |
| Django     | 6.0.5  | Веб-фреймворк      | BSD (відкрита) |

|                 |         |                            |                |
|-----------------|---------|----------------------------|----------------|
|                 |         | MTV/MVC                    |                |
| SQLite          | 3.45+   | Реляційна СУБД             | Public Domain  |
| Bootstrap       | 5.3     | CSS-фреймворк та UI        | MIT (відкрита) |
| Leaflet.js      | 1.9.x   | Інтерактивна картографія   | BSD (відкрита) |
| Faker           | 40.15.0 | Генерація тестових даних   | MIT (відкрита) |
| Bootstrap Icons | 1.11+   | Іконки інтерфейсу          | MIT (відкрита) |
| PyCharm Pro     | 2025.x  | IDE                        | Комерційна     |
| Git / GitHub    | 2.x     | Контроль версій            | GPL (відкрита) |
| Nginx           | 1.24+   | Реверс-проксі (production) | BSD (відкрита) |
| Gunicorn        | 21.x    | WSGI-сервер                | MIT (відкрита) |

*Таблиця 2.1 — Технологічний стек системи UIC Manager*

## 2.2. Проєктування архітектури системи

### 2.2.1. Загальна архітектура (MVC/MTV)

Система UIC Manager побудована за шаблоном MTV (Model-Template-View), що є реалізацією MVC у Django. Компоненти:

- Model (Модель) — файл `infrastructure/models.py`. Визначає структуру даних у вигляді Python-класів. Не залежить від способу відображення. Django ORM автоматично трансліює операції над об'єктами у SQL-запити.
- Template (Шаблон) — HTML-файли у директорії `templates/`. Отримують контекст від View і генерують HTML-відповідь. Містять логіку відображення (цикли, умови), але не бізнес-логіку.

- View (Представлення) — файли `views.py` та `views_billing.py`. Функції-обробники HTTP-запитів. Приймають Request, звертаються до моделей, формують контекст і повертають Response з відрендереним шаблоном.

Рівні архітектури:

Рівень представлення: веб-браузер, що виконує HTTP-запити та відображає HTML. Bootstrap 5 — адаптивна верстка. Leaflet.js — інтерактивна карта. Vanilla JavaScript — динамічна поведінка форм.

Рівень застосунку: Django-застосунок. URL-роутер направляє запити до View. View-функції реалізують бізнес-логіку та взаємодіють з моделями. Django Middleware обробляє наскрізні аспекти: аутентифікацію, CSRF-захист, сесії.

Рівень даних: SQLite, що зберігає всі дані системи. Django ORM забезпечує абстракцію доступу та захист від SQL-ін'єкцій через параметризацію всіх запитів.

### 2.2.2. Структура проєкту

| Шлях                                  | Опис                                                                                                                                        |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>core/settings.py</code>         | Глобальна конфігурація: <code>INSTALLED_APPS</code> , <code>DATABASES</code> , <code>MIDDLEWARE</code> , <code>TEMPLATES</code> , час. пояс |
| <code>core/urls.py</code>             | Кореневий маршрутизатор — підключає <code>infrastructure/urls.py</code> та <code>admin/</code>                                              |
| <code>infrastructure/models.py</code> | 15 ORM-моделей, сигнали, функції                                                                                                            |

|                                                             |                                                                                 |
|-------------------------------------------------------------|---------------------------------------------------------------------------------|
|                                                             | генерації                                                                       |
| infrastructure/views.py                                     | ~800 рядків:<br>dashboard, clients,<br>equipment, tasks,<br>warehouse тощо      |
| infrastructure/views_billing.py                             | ~600 рядків: billing,<br>topup, charge,<br>promised,<br>block/unblock           |
| infrastructure/admin.py                                     | Конфігурація<br>Django Admin:<br>list_display,<br>search_fields,<br>list_filter |
| infrastructure/migrations/0001–0029                         | 29 міграцій схеми<br>БД — повна<br>еволюція від<br>початку                      |
| infrastructure/management/commands/monthly_charge.py        | Автоматичне<br>місячне списання                                                 |
| infrastructure/management/commands/generate_billing_data.py | Генератор тестових<br>даних Faker                                               |
| templates/base.html                                         | Базовий шаблон:<br>sidebar, навігація,<br>Bootstrap 5 CDN, 32<br>761 байт       |
| templates/dashboard.html                                    | Дашборд: картки<br>статистики, останні                                          |

|                               |                                                     |
|-------------------------------|-----------------------------------------------------|
|                               | записи                                              |
| templates/billing.html        | Білінг: таблиця балансів, фільтри, статистика       |
| templates/billing_client.html | Деталі білінгу клієнта: рахунок, журнал операцій    |
| templates/clients.html        | Список клієнтів із пагінацією та пошуком            |
| templates/client_detail.html  | Картка клієнта: усі дані, тікети, операції          |
| templates/network_map.html    | Інтерактивна карта Leaflet.js + OpenStreetMap       |
| templates/tasks.html          | Задачі, заявки, аварійні задачі                     |
| db.sqlite3                    | Файл бази даних (1 363 968 байт з тестовими даними) |

Таблиця 2.2 — Структура проєкту UIC Manager

### 2.2.3. Маршрутизація URL

| URL       | View-функція   | Методи | Призначення                   |
|-----------|----------------|--------|-------------------------------|
| /         | dashboard_view | GET    | Головна панель управління     |
| /clients/ | clients_list   | GET,   | Список та реєстрація клієнтів |

|                    |                |              |                             |
|--------------------|----------------|--------------|-----------------------------|
|                    |                | POST         |                             |
| /clients/<id>/     | client_detail  | GET,<br>POST | Деталі клієнта              |
| /billing/          | billing_view   | GET          | Білінговий модуль           |
| /billing/topup/    | topup_view     | POST         | Поповнення рахунку          |
| /billing/charge/   | charge_view    | POST         | Ручне списання              |
| /billing/promised/ | promised_view  | POST         | Обіцяний платіж             |
| /tasks/            | tasks_view     | GET,<br>POST | Задачі та заявки            |
| /equipment/        | equipment_list | GET,<br>POST | Список обладнання           |
| /network-map/      | network_map    | GET          | Інтерактивна мережева карта |
| /tariffs/          | tariffs_view   | GET,<br>POST | Тарифні плани               |
| /warehouse/        | warehouse_view | GET,<br>POST | Склад та магазин            |
| /personnel/        | personnel_view | GET,<br>POST | Управління персоналом       |
| /messages/         | messages_view  | GET,<br>POST | Повідомлення клієнтам       |
| /search/           | search_view    | GET          | Глобальний пошук            |
| /login/            | login_view     | GET,<br>POST | Авторизація в системі       |
| /logout/           | logout_view    | POST         | Вихід з системи             |

Таблиця 2.3 — URL-маршрутизація системи UIC Manager

## 2.3. Моделювання та проєктування бази даних

### 2.3.1. Ключові таблиці

Розглянемо детальну структуру ключових таблиць БД системи UIC Manager.

| Поле        | Тип даних    | Обмеження                 | Опис                         |
|-------------|--------------|---------------------------|------------------------------|
| id          | INTEGER      | PRIMARY KEY,<br>AI        | Унікальний ідентифікатор     |
| name        | VARCHAR(100) | NOT NULL                  | Назва обладнання             |
| ip_address  | INET         | NOT NULL,<br>UNIQUE       | IPv4 або IPv6 адреса         |
| mac_address | VARCHAR(17)  | NULL, UNIQUE              | MAC:<br>XX:XX:XX:XX:XX:XX    |
| location    | VARCHAR(255) | NOT NULL                  | Адреса розташування вузла    |
| is_active   | BOOLEAN      | NOT NULL,<br>DEFAULT=True | Статус онлайн/офлайн         |
| latitude    | DECIMAL(9,6) | NULL                      | Широта (напр.<br>50.176534)  |
| longitude   | DECIMAL(9,6) | NULL                      | Довгота (напр.<br>30.307621) |

Таблиця 2.4 — Структура таблиці *Equipment* (обладнання)

| Поле      | Тип даних    | Обмеження          | Опис                     |
|-----------|--------------|--------------------|--------------------------|
| id        | INTEGER      | PRIMARY KEY,<br>AI | Унікальний ідентифікатор |
| full_name | VARCHAR(150) | NOT NULL           | ПІБ абонента             |

|                 |              |                            |                    |
|-----------------|--------------|----------------------------|--------------------|
| contract_number | VARCHAR(20)  | NOT NULL,<br>UNIQUE        | Номер договору     |
| onu_serial      | VARCHAR(20)  | NULL, UNIQUE               | Серійний номер ONU |
| address         | VARCHAR(255) | NOT NULL                   | Адреса підключення |
| phone           | VARCHAR(20)  | NOT NULL                   | Контактний телефон |
| equipment_id    | INTEGER      | FK→Equipment,<br>NULL      | Вузол підключення  |
| tariff_id       | INTEGER      | FK→Tariff,<br>NULL         | Тарифний план      |
| is_online       | BOOLEAN      | NOT NULL,<br>DEFAULT=False | Онлайн статус      |
| is_blocked      | BOOLEAN      | NOT NULL,<br>DEFAULT=False | Статус блокування  |

Таблиця 2.5 — Структура таблиці Client (абонент)

| Поле       | Тип                             | Опис                                        |
|------------|---------------------------------|---------------------------------------------|
| id         | INTEGER PK                      | Ідентифікатор                               |
| client_id  | FK→Client, OneToOne,<br>CASCADE | Прив'язка до клієнта (один до одного)       |
| balance    | DECIMAL(10,2)                   | Поточний баланс рахунку (грн)               |
| updated_at | DATETIME                        | Час останнього оновлення<br>(auto_now=True) |

Таблиця 2.6 — Структура таблиці Account (рахунок)

| Поле      | Тип        | Опис                                    |
|-----------|------------|-----------------------------------------|
| id        | INTEGER PK | Ідентифікатор                           |
| client_id | FK→Client, | Прив'язка до клієнта (один до багатьох) |



|              |               |                                      |
|--------------|---------------|--------------------------------------|
|              | CASCADE       |                                      |
| amount       | DECIMAL(10,2) | Сума операції (грн)                  |
| payment_type | VARCHAR(20)   | Тип: 'topup' / 'charge' / 'promised' |
| comment      | VARCHAR(255)  | Коментар оператора                   |
| created_by   | VARCHAR(100)  | Ім'я оператора, що провів операцію   |
| created_at   | DATETIME      | Час операції (auto_now_add=True)     |

Таблиця 2.7 — Структура таблиці *Payment* (платіж)

### 2.3.2. Схема зв'язків між таблицями

- Client → Equipment: ForeignKey M:1, on\_delete=SET\_NULL — видалення вузла не знищує клієнтів;
- Client → Tariff: ForeignKey M:1, on\_delete=SET\_NULL — зміна тарифів не впливає на записи клієнтів;
- Account → Client: OneToOneField 1:1, on\_delete=CASCADE — рахунок видаляється разом з клієнтом;
- ClientCredentials → Client: OneToOneField 1:1, on\_delete=CASCADE — облікові дані видаляються разом з клієнтом;
- Payment → Client: ForeignKey M:1, on\_delete=CASCADE — платежі видаляються разом з клієнтом;
- Ticket → Client: ForeignKey M:1, on\_delete=CASCADE — тікети прив'язані до клієнта;
- ConnectionApplication → Employee: ForeignKey M:1, on\_delete=SET\_NULL — виконавець необов'язковий;
- EmergencyTask → Employee: ManyToManyField M:M — до аварії може бути призначено кілька фахівців.

Автоматична генерація облікових даних реалізована через Django-сигнали.

Декоратор `@receiver(post_save, sender=Client)` гарантує виклик функції

`create_client_credentials` після кожного збереження нового об'єкта `Client` (параметр

created=True).

Система UIC Manager налічує 29 міграцій бази даних, що відображають еволюцію схеми від першої версії до поточної. Кожна зміна моделей Python автоматично фіксується командою `python manage.py makemigrations` і застосовується до БД командою `python manage.py migrate`.

## **2.4. Опис алгоритмів роботи системи**

### **2.4.1. Алгоритм реєстрації нового клієнта**

1. Оператор відкриває форму реєстрації нового клієнта.
2. Вводить: ПІБ, номер договору, адресу, телефон, серійний номер ONU (опційно), тарифний план, вузол обладнання.
3. Серверна валідація: перевірка унікальності `contract_number` та `onu_serial` через ORM. При дублюванні — повернення форми з повідомленням про помилку.
4. Збереження запису Client у БД: `Client.objects.create(**cleaned_data)`.
5. Автоматичний виклик Django-сигналу `post_save` → `create_client_credentials()`. Функція `generate_login()` циклічно генерує 7-значний числовий логін до досягнення унікальності. Функція `generate_password()` генерує пароль у форматі «буква + 4 цифри».
6. Автоматичне створення рахунку Account з початковим балансом 0.00 грн.
7. Перенаправлення оператора на сторінку деталей клієнта з відображенням облікових даних.

### **2.4.2. Алгоритм місячного нарахування (monthly\_charge)**

8. Запуск: `python manage.py monthly_charge` (вручну або через cron 1-го числа місяця).
9. Отримання всіх клієнтів з тарифним планом:  
`Client.objects.filter(tariff__isnull=False).select_related('tariff', 'account')`.
10. Для кожного клієнта:

а) Перевірка активного обіцяного платежу:

`PromisedPayment.objects.filter(client=client, is_active=True, expires_at__gt=now())`.

Якщо є — пропустити клієнта (`continue`).

б) Списання: `account.balance -= tariff.price; account.save()`.

в) Якщо `account.balance < 0` та `not client.is_blocked` → `client.is_blocked = True; client.save()`.

г) Фіксація операції: `Payment.objects.create(client=client, amount=tariff.price, payment_type='charge', comment='Щомісячне нарахування')`.

11. Виведення звіту: оброблено  $X$  клієнтів, заблоковано  $Y$ , загальна сума  $Z$  грн.

### 2.4.3. Алгоритм поповнення рахунку та розблокування

12. Оператор вводить суму поповнення та необов'язковий коментар через форму.

13. Валідація: `amount > 0`, інакше — помилка.

14. Атомарна транзакція: `account.balance += amount; account.save()`.

15. Якщо `client.is_blocked` та `account.balance >= 0` → `client.is_blocked = False; client.save()` — автоматичне розблокування.

16. `Payment.objects.create(client=client, amount=amount, payment_type='topup', comment=comment, created_by=request.user.username)`.

17. Перенаправлення з повідомленням про успішну операцію.

### 2.4.4. Алгоритм глобального пошуку

18. Отримання `query` з GET-параметра `q`. Якщо довжина  $< 2$  символів — повернення порожньої сторінки.

19. Пошук по клієнтах: `Client.objects.filter(Q(full_name__icontains=query) | Q(contract_number__icontains=query) | Q(phone__icontains=query) | Q(address__icontains=query) | Q(onu_serial__icontains=query))`.

20. Пошук по обладнанню: `Equipment.objects.filter(Q(name__icontains=query) | Q(ip_address__icontains=query) | Q(mac_address__icontains=query) | Q(location__icontains=query))`.
21. Передача результатів у контекст шаблону: `{'clients': clients_qs, 'equipment': equipment_qs, 'query': query}`.
22. Шаблон відображає розділені секції результатів з посиланнями на відповідні записи та загальну кількість збігів.

#### **2.4.5. Алгоритм реєстрації аварійної задачі**

23. Оператор відкриває форму реєстрації аварії (Emergency Task Form).
24. Вводить: місто (за замовчуванням м. Васильків), вулицю, повну адресу, опис проблеми, координати.
25. Вибирає одного або кількох виконавців зі списку активних співробітників.
26. Система зберігає `EmergencyTask` зі статусом 'new'. Через `ManyToMany`: `task.assignees.set(selected_employees)`.
27. Задача з'являється на карті з помаранчевим маркером аварії.
28. Виконавці вирушають на місце, після усунення — оновлюють статус на 'done'.

### **2.5. Реалізація програмного продукту**

#### **2.5.1. Реалізація моделей (models.py)**

Розглянемо ключові особливості реалізації ORM-моделей:

Модель `Equipment` використовує `GenericIPAddressField` для IP-адреси, що забезпечує вбудовану валідацію формату IPv4/IPv6 без додаткових перевірок. `DecimalField` з `decimal_places=6` для координат — точність близько 0.1 м, достатня для геолокації.

Автогенерація облікових даних:

```
def generate_login():  
    while True:
```

```

login = ".join(random.choices(string.digits, k=7))
if not ClientCredentials.objects.filter(login=login).exists():
    return login

```

```

@receiver(post_save, sender=Client)
def create_client_credentials(sender, instance, created, **kwargs):
    if created: # Тільки при першому збереженні!
        ClientCredentials.objects.get_or_create(client=instance)

```

Параметр `created=True` гарантує, що облікові дані генеруються виключно при першій реєстрації клієнта, а не при кожному оновленні запису.

### 2.5.2. Реалізація View-функцій

Функція `dashboard_view` — агрегація статистики через ORM:

```

def dashboard_view(request):
    context = {
        'clients_total': Client.objects.count(),
        'clients_active': Client.objects.filter(is_blocked=False).count(),
        'equipment_online': Equipment.objects.filter(is_active=True).count(),
        'new_tickets': Ticket.objects.filter(status='new').count(),
        'recent_clients': Client.objects.order_by('-id')[:5],
    }
    return render(request, 'dashboard.html', context)

```

Функція `billing_view` демонструє фільтрацію та оптимізацію запитів:

```

def billing_view(request):
    filter_type = request.GET.get('filter', 'all')
    clients = Client.objects.select_related('account', 'tariff')
    if filter_type == 'negative':
        clients = clients.filter(account__balance__lt=0)

```

```
elif filter_type == 'blocked':

    clients = clients.filter(is_blocked=True)
```

Метод `select_related` виконує SQL JOIN замість окремих запитів для кожного об'єкта, скорочуючи кількість SQL-запитів з  $N+1$  до 1 при відображенні списку клієнтів з рахунками.

### 2.5.3. Реалізація шаблону `network_map.html`

Інтеграція Leaflet.js з Django-шаблоном:

```
var map = L.map('map').setView([50.1761, 30.3079], 13);

L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '© OpenStreetMap contributors'
}).addTo(map);

{% for eq in equipment_list %}

    {% if eq.latitude and eq.longitude %}

        var marker = L.marker([{{ eq.latitude }}, {{ eq.longitude }}], {
            icon: {% if eq.is_active %}greenIcon{% else %}redIcon{% endif %}
        }).bindPopup('<b>{{ eq.name }}</b><br>IP: {{ eq.ip_address }}')
        .addTo(map);

    {% endif %}

{% endfor %}
```

Координати обладнання передаються з View у контексті шаблону та вставляються безпосередньо у JavaScript-код. Django Template Language автоматично екранує спеціальні символи для захисту від XSS.

### 2.5.4. Management-команда `monthly_charge`

Django management-команди — потужний механізм для виконання серверних задач за розкладом або вручну. Команда `monthly_charge` (`infrastructure/management/commands/monthly_charge.py`) виконує автоматичне місячне нарахування:

```
from django.core.management.base import BaseCommand
```

```

from django.utils import timezone

class Command(BaseCommand):
    help = 'Щомісячне нарахування абонентської плати'

    def handle(self, *args, **options):
        clients = Client.objects.filter(
            tariff__isnull=False).select_related('account','tariff')
        charged, blocked = 0, 0
        for client in clients:
            if PromisedPayment.objects.filter(
                client=client, is_active=True,
                expires_at__gt=timezone.now()).exists():
                continue # Пропустити — є обіцяний платіж
            account = client.account
            account.balance -= client.tariff.price
            account.save()
            if account.balance < 0 and not client.is_blocked:
                client.is_blocked = True
                client.save()
                blocked += 1
            Payment.objects.create(client=client,
                amount=client.tariff.price, payment_type='charge',
                comment='Щомісячне нарахування')
            charged += 1

        self.stdout.write(f'Оброблено: {charged}, заблоковано: {blocked}')

```

## 2.6. Інтерфейс користувача

### 2.6.1. Концепція дизайну

Інтерфейс UIC Manager розроблено з урахуванням принципів UX для

корпоративних застосунків: ефективність (мінімум кліків для частих операцій), передбачуваність (однаковий розклад елементів на всіх сторінках), інформативність (контекстуальна підсвітка критичних станів).

Колірна схема: темна бічна панель (#1a1a2e, #16213e) з білим текстом для чіткого розмежування навігації від контенту. Основний фон контентної зони — світло-сірий (#f8f9fa) для зниження навантаження на зір. Акцентний колір — Bootstrap Primary (#0d6efd) для кнопок та активних елементів. Критичні стани (від'ємний баланс, заблоковані клієнти) — Bootstrap Danger (#dc3545).

## 2.6.2. Опис екранів системи

Головна панель (Dashboard) відображає 6 карток зі статистикою (клієнти загально/активні/заблоковані, обладнання онлайн/офлайн, нові тікети) та таблиці останніх подій. Дозволяє оператору за секунди оцінити загальний стан ISP.

Сторінка клієнтів (Clients) відображає таблицю з пагінацією (50 записів), панелью фільтрів (всі/активні/заблоковані) та рядком пошуку. Кнопка «Додати клієнта» відкриває форму реєстрації.

Картка клієнта (Client Detail) містить блоки: основна інформація, фінансовий стан (баланс з кольоровим кодуванням), кнопки операцій (поповнити/заблокувати/обіцяний платіж), тарифний план, облікові дані RADIUS, журнал платежів, список тікетів.

Сторінка білінгу (Billing) відображає таблицю клієнтів з балансами, підсвіченими кольором: зелений — баланс  $\geq$  тарифу, жовтий — між 0 і тарифом, червоний — від'ємний або заблокований. Бокова панель: агрегована статистика (загальний баланс по всіх клієнтах, сума боргів).

Мережева карта (Network Map) — Leaflet.js/OpenStreetMap. Маркери: зелені (онлайн), червоні (офлайн), помаранчеві (аварії). Роруп-вікно з деталями при кліку.

Склад (Warehouse) — двовкладкова структура: «Склад» (матеріали для монтажу з категоріями та одиницями виміру) та «Магазин» (товари для продажу клієнтам з фото та цінами у вигляді карток Bootstrap Card).



Задачі (Tasks) — три секції: «Заявки на підключення», «Тікети клієнтів», «Аварійні задачі». Кожна задача має статус з кольоровим бейджем, пріоритет та кнопки зміни статусу.

| Шаблон              | Розмір      | Ключові елементи                                                    |
|---------------------|-------------|---------------------------------------------------------------------|
| base.html           | 32 761 байт | Sidebar, Bootstrap 5 CDN, Bootstrap Icons, навігація з лічильниками |
| dashboard.html      | 15 447 байт | 6 статистичних карток, 3 таблиці останніх подій                     |
| billing.html        | 13 859 байт | Таблиця з балансами, кольорове кодування, агрегована статистика     |
| billing_client.html | 26 126 байт | Деталі рахунку, журнал операцій, форми операцій                     |
| clients.html        | 4 022 байт  | Таблиця з пагінацією, фільтри, пошук                                |
| client_detail.html  | 4 394 байт  | Повна картка клієнта, блоки даних                                   |
| network_map.html    | 3 786 байт  | Leaflet.js, маркери обладнання та аварій                            |
| tasks.html          | 16 128 байт | Три секції задач, форми реєстрації, статуси                         |
| warehouse.html      | 6 479 байт  | Двовкладкова структура склад/магазин                                |
| tariffs.html        | 4 922 байт  | Картки тарифів з деталями опцій                                     |

Таблиця 2.8 — Шаблони системи *UIC Manager*

## 2.7. Розгортання системи

Система UIC Manager розгорнута у двох режимах: локальному (для розробки та тестування) та production-режимі через хмарну платформу PythonAnywhere.

Локальне розгортання. Для запуску системи локально необхідно виконати команду `python manage.py runserver` у директорії проєкту. Система стає доступною у браузері за адресою `http://localhost:8000/`. Тестова база даних наповнюється

командою `python manage.py generate_billing_data`, що генерує реалістичні дані з реальними вулицями міста Васильків та навколишніх сіл.

Розгортання на PythonAnywhere. PythonAnywhere — хмарна платформа для хостингу Python-застосунків, що підтримує Django без необхідності налаштування власного сервера. Система UIC Manager успішно розгорнута та доступна за адресою: <https://shtorm96.pythonanywhere.com/>

### **Кроки розгортання на PythonAnywhere:**

1. Реєстрація облікового запису на [pythonanywhere.com](https://pythonanywhere.com) (безкоштовний тариф підходить для демонстрації).
2. Відкрити консоль PythonAnywhere → Bash та клонувати репозиторій з GitHub: `git clone https://github.com/shtorm96/UICMANAGER`.
3. Створити `virtualenv` та встановити залежності: `pip install -r requirements.txt`.
4. Виконати міграції бази даних: `python manage.py migrate`.
5. У розділі "Web" панелі PythonAnywhere налаштувати WSGI-файл для під'єднання Django-застосунку.
6. Налаштувати роздачу статичних файлів через Static files → URL: `/static/`, Directory: `/path/to/staticfiles/`.
7. Оновити `ALLOWED_HOSTS` у `settings.py`: `['shtorm96.pythonanywhere.com']`.
8. Натиснути кнопку "Reload" для перезапуску веб-сервера.

Репозиторій проєкту на GitHub забезпечує контроль версій та спрощує розгортання оновлень. Для оновлення production-версії достатньо виконати `git pull origin main` у консолі PythonAnywhere та перезапустити сервер.

### **Переваги PythonAnywhere для даного проєкту:**

Безкоштовний тариф включає постійно активний веб-сервер, доступний за URL виду `ім'я.pythonanywhere.com`; вбудована підтримка Python та Django без налаштування Nginx/Gunicorn; зручна веб-консоль для виконання management-команд; автоматичне керування процесами.

База даних SQLite зберігається у файловій системі PythonAnywhere та не потребує окремого сервера СУБД. Оновлення бази здійснюється стандартними Django-міграціями.

## РОЗДІЛ III. ТЕСТУВАННЯ ТА ВЕРИФІКАЦІЯ

### 3.1. Методи тестування

Систему я перевіряв на кількох рівнях — від простих перевірок окремих функцій до перевірки роботи цілих сценаріїв. Нижче описано, які методи тестування застосовувались і що саме перевірялось.

#### 3.1.1. Модульне тестування (Unit Testing)

Перевіряє окремі функції та класи в ізоляції від інших компонентів. Використовується `django.test.TestCase` (базується на Python `unittest`). Django автоматично створює ізольовану тестову БД для кожного запуску тестів, очищаючи її після завершення.

Тестувались:

- функції `generate_login()` та `generate_password()` — перевірка формату та унікальності;
- метод `save()` моделі `ClientCredentials` — умовна генерація (тільки при порожніх полях);
- сигнал `create_client_credentials` — автоматичне створення при реєстрації клієнта;
- логіка `monthly_charge` — сценарії: позитивний баланс (без блокування), від'ємний (з блокуванням), наявність обіцяного платежу (пропуск клієнта).

Приклад unit-тесту:

- `from django.test import TestCase`
- `from infrastructure.models import Client, ClientCredentials, Tariff`
- `class ClientSignalTest(TestCase):`

|             |  |             |        |      |               |  |  |                   |      |        |    |
|-------------|--|-------------|--------|------|---------------|--|--|-------------------|------|--------|----|
|             |  |             |        |      | 641.23.26. ДП |  |  |                   |      |        |    |
|             |  |             |        |      |               |  |  |                   |      |        |    |
| Зм.         |  | № докум.    | Підпис | Дата |               |  |  |                   |      |        |    |
| Розроб.     |  | Тромса Д.С. |        |      | Розділ 3      |  |  | Літ.              | Лист | Листів |    |
| Перевір.    |  | Галич Н.С   |        |      |               |  |  |                   |      | 43     | 66 |
| Консультант |  |             |        |      |               |  |  | ВФК ДНП «ДУ «КАІ» |      |        |    |
| Н. Контр.   |  |             |        |      |               |  |  |                   |      |        |    |
| Затверд.    |  |             |        |      |               |  |  |                   |      |        |    |

```

def test_credentials_auto_created(self):
    tariff = Tariff.objects.create(name='Базовий', price=250, speed=100)
    client = Client.objects.create(
        full_name='Тест Тест Тестович',
        contract_number='00001', address='вул. Тестова, 1',
        phone='0501234567', tariff=tariff
    )
    creds = ClientCredentials.objects.get(client=client)
    self.assertEqual(len(creds.login), 7)
    self.assertTrue(creds.login.isdigit())
    self.assertEqual(len(creds.password), 5)

```

### 3.1.2. Функціональне та інтеграційне тестування

Функціональне тестування перевіряє відповідність системи функціональним вимогам методом «чорного ящика». Використовується Django Test Client для відправки HTTP-запитів та перевірки відповідей:

```

from django.test import TestCase, Client as TestClient
class BillingViewTest(TestCase):
    def setUp(self):
        # Створення тестових даних
        self.admin = User.objects.create_superuser('admin', '', 'pass')
        self.tc = TestClient()
        self.tc.login(username='admin', password='pass')
    def test_topup_increases_balance(self):
        # Перевірка: поповнення збільшує баланс
        resp = self.tc.post('/billing/topup/',
            {'client_id': self.client.id, 'amount': 300})
        self.client.account.refresh_from_db()

```

```
self.assertEqual(self.client.account.balance, 300)
```

Інтеграційне тестування перевіряє взаємодію компонентів: повний цикл від POST-запиту реєстрації клієнта до відображення облікових даних на сторінці деталей.

### 3.1.3. UI/UX тестування

Тестування інтерфейсу проводилось вручну на реальних пристроях:

- Desktop: Chrome 124, Firefox 126, Edge 124 на Windows 11;
- Mobile: Chrome for Android (роздільна здатність 1080×2340, CSS-ширина 393px);
- Tablet: Safari на iPad (ширина 768px).

Перевірялись: коректність верстки таблиць та форм, адаптивність sidebar, кольорове кодування балансів, функціональність модальних вікон.

### 3.1.4. Навантажувальне тестування

Apache Benchmark для вимірювання продуктивності при 50 одночасних запитах:

```
ab -n 1000 -c 50 -H 'Cookie: sessionid=<value>' http://localhost:8000/clients/
```

Django Debug Toolbar для вимірювання кількості SQL-запитів та часу відповіді.

Бібліотека Faker та management-команда `generate_billing_data` для наповнення БД тестовими даними: 1000, 3000 та 5000 клієнтів.

### 3.1.5. Тестування безпеки

- SQL-ін'єкція: Django ORM параметризує всі запити, виключаючи можливість SQL-ін'єкції;
- XSS (Cross-Site Scripting): Django Template Language автоматично екранує виведення змінних;
- CSRF (Cross-Site Request Forgery): Django CSRF Middleware перевіряє токен для всіх POST-запитів;

- Несанкціонований доступ: декоратор `@login_required` на всіх View перенаправляє неаутентифікованих на сторінку логіну;
- Форс-перегляд URL: перевірено пряму адресацію захищених URL без авторизації.

### 3.2. Розробка тестових сценаріїв

| ID    | Сценарій                                 | Передумови                            | Очікуваний результат                                                 |
|-------|------------------------------------------|---------------------------------------|----------------------------------------------------------------------|
| ТС-01 | Реєстрація нового клієнта (позитивний)   | Тарифи та обладнання існують          | Клієнт збережений; автоматично створено Account та ClientCredentials |
| ТС-02 | Реєстрація клієнта (дублікат договору)   | Клієнт з таким договором існує        | Помилка валідації; запис не створено                                 |
| ТС-03 | Поповнення рахунку активного клієнта     | Клієнт з рахунком існує               | Баланс зріс; Payment записано з типом 'topup'                        |
| ТС-04 | Поповнення рахунку заблокованого клієнта | Клієнт заблокований, баланс від'ємний | Баланс зріс; is_blocked=False; Payment записано                      |
| ТС-05 | Місячне списання — баланс > тарифу       | Баланс 500 грн, тариф 250 грн         | Баланс 250 грн; клієнт не заблокований                               |
| ТС-06 | Місячне списання — баланс < тарифу       | Баланс 100 грн, тариф 250 грн         | Баланс -150 грн; is_blocked=True                                     |

|       |                                    |                                         |                                                         |
|-------|------------------------------------|-----------------------------------------|---------------------------------------------------------|
| ТС-07 | Місячне списання — обіцяний платіж | Клієнт з активним PromisedPayment       | Клієнт пропущений; баланс не змінено                    |
| ТС-08 | Обіцяний платіж для заблокованого  | Клієнт заблокований                     | PromisedPayment створено; is_blocked=False              |
| ТС-09 | Реєстрація заявки на підключення   | Оператор авторизований, є співробітники | ConnectionApplication зі статусом 'new'                 |
| ТС-10 | Реєстрація аварії з координатами   | Є активні співробітники                 | EmergencyTask збережено; маркер на карті                |
| ТС-11 | Глобальний пошук за ПІБ клієнта    | Клієнти існують у БД                    | Відображено релевантні результати з посиланнями         |
| ТС-12 | Видалення обладнання з клієнтами   | Обладнання має прив'язаних клієнтів     | Обладнання видалено; клієнти збережені з equipment=NULL |

*Таблиця 3.1 — Тестові сценарії системи UIC Manager*

Тестові сценарії для граничних умов:

- Поповнення рахунку на від'ємну суму — помилка валідації (сума має бути > 0);
- Обіцяний платіж клієнту з вже активним обіцяним — попередження оператора;
- Реєстрація обладнання з некоректним форматом IP — помилка GenericIPAddressField;
- Доступ до всіх URL без авторизації — перенаправлення на /login/;

– Пошук з рядком < 2 символів — порожня сторінка результатів.

### 3.3. Аналіз результатів тестування

Усього виконано 76 тестових перевірок за 6 категоріями. З них успішно пройшли 73, виявлено 7 дефектів, які було усунено. Детальні результати наведено в таблиці нижче.

| Категорія тестування             | Тестів     | Успішно | Дефектів | Усунено |
|----------------------------------|------------|---------|----------|---------|
| Модульне (Unit)                  | 8          | 8/8     | 0        | —       |
| Функціональне (основні сценарії) | 12         | 11/12   | 3        | 3       |
| Функціональне (граничні умови)   | 8          | 7/8     | 2        | 2       |
| Інтеграційне                     | 12         | 12/12   | 0        | —       |
| UI/UX (Desktop)                  | 15 екранів | 15/15   | 1        | 1       |
| UI/UX (Mobile)                   | 15 екранів | 14/15   | 1        | 1       |
| Безпека                          | 6          | 6/6     | 0        | —       |
| Разом                            | 76+        | 73/76   | 7        | 7       |

Таблиця 3.2 — Зведені результати тестування

| ID   | Опис дефекту                                                                     | Тип                                | Статус  |
|------|----------------------------------------------------------------------------------|------------------------------------|---------|
| D-01 | При поповненні рахунку заблокованого клієнта блокування не знімалось автоматично | Логічна помилка у views_billing.py | Усунено |



|      |                                                                                     |                                    |                                   |
|------|-------------------------------------------------------------------------------------|------------------------------------|-----------------------------------|
| D-02 | Фільтр 'Заблоковані' у білінгу не враховував від'ємний баланс без явного блокування | Логічна помилка фільтра ORM        | Усунено                           |
| D-03 | Тікети клієнта не завантажувались при першому відкритті картки (проблема N+1)       | Проблема продуктивності ORM        | Усунено через prefetch_related    |
| D-04 | Розповзання верстки таблиці білінгу у Firefox 126                                   | CSS-несумісність                   | Усунено (додано overflow-x: auto) |
| D-05 | Помилка збереження обладнання з IPv6-адресою у певному форматі                      | Конфігурація GenericIPAddressField | Усунено (protocol='both')         |
| D-06 | Некоректна пагінація при фільтрі 'від'ємний баланс'                                 | Логічна помилка в пагінаторі       | Усунено                           |
| D-07 | Відображення мобільного меню перекривало таблицю на ширині 380-400px                | CSS адаптивність                   | Усунено (z-index)                 |

*Таблиця 3.3 — Виявлені та усунені дефекти*

### 3.4. Оцінка продуктивності системи

| Сторінка / операція           | 100 кл. | 1000 кл. | 3000 кл. | SQL-запитів |
|-------------------------------|---------|----------|----------|-------------|
| Dashboard                     | 42 мс   | 95 мс    | 185 мс   | 8           |
| Список клієнтів<br>(50/стор.) | 28 мс   | 35 мс    | 62 мс    | 3           |
| Картка клієнта<br>(детальна)  | 24 мс   | 26 мс    | 29 мс    | 7           |
| Білінг (з пагінацією)         | 35 мс   | 48 мс    | 75 мс    | 5           |
| Глобальний пошук              | 18 мс   | 55 мс    | 88 мс    | 4           |
| Мережева карта                | 38 мс   | 42 мс    | 58 мс    | 5           |
| Поповнення рахунку<br>(POST)  | 12 мс   | 14 мс    | 16 мс    | 3           |

Таблиця 3.4 — Результати вимірювання часу відповіді

Система відповідає вимозі НФВ-1 ( $\leq 2000$  мс) з великим запасом. Найбільший час відповіді — 185 мс на Dashboard при 3000 клієнтів, що в 10 разів нижче встановленого порогу.

| Метрика                  | Список клієнтів | Дашборд | Білінг (стор. 1) |
|--------------------------|-----------------|---------|------------------|
| Запитів за секунду (RPS) | 284             | 198     | 156              |
| Середній час відповіді   | 176 мс          | 253 мс  | 321 мс           |
| 95-й перцентиль          | 285 мс          | 412 мс  | 498 мс           |
| Помилки (з 1000 запитів) | 0               | 0       | 0                |

Таблиця 3.5 — Результати навантажувального тестування (50 одночасних запитів)

Система стабільно обробляє 50 одночасних запитів без помилок. Для типового регіонального провайдера (3–10 одночасних операторів) час відклику буде суттєво нижчим.

### 3.5. Порівняння з аналогами

| Критерій                      | UIC Manager   | WHMCS      | UTM5        | Zabbix     |
|-------------------------------|---------------|------------|-------------|------------|
| Реєстр обладнання + карта     | Повний        | Відсутній  | Відсутній   | Моніторинг |
| Облік абонентів               | Повний (укр.) | Частковий  | Частковий   | Відсутній  |
| Білінг + обіцяні платежі      | Повний        | Частковий  | Повний      | Відсутній  |
| Тікети / заявки               | Є             | Є          | Немає       | Немає      |
| Аварійні задачі з геолокацією | Є             | Немає      | Немає       | Частково   |
| Складський облік              | Є             | Немає      | Немає       | Немає      |
| Управління персоналом         | Є             | Немає      | Немає       | Немає      |
| Час розгортання               | ~2 год        | ~4 год     | ~8 год      | ~12 год    |
| Ціна (рік)                    | 0 грн         | ~9 840 грн | ~20 500 грн | 0 грн      |
| Україномовний інтерфейс       | Так           | Ні         | Ні          | Ні         |

Таблиця 3.6 — Порівняльний аналіз UIC Manager з аналогами

UIC Manager є єдиним серед розглянутих рішень, що об'єднує в одній системі технічне управління обладнанням (з інтерактивною картою), бізнес-функції (білінг, клієнти, персонал) та операційні функції (аварії, тікети, склад) з повною українською локалізацією та нульовою вартістю ліцензій.

### **3.7. Рекомендації щодо впровадження**

#### **3.7.1. Порядок розгортання**

29. Встановити Python 3.12: `sudo apt update && sudo apt install python3.12 python3.12-venv`.
30. Клонувати репозиторій та налаштувати Python virtualenv.
31. Встановити залежності: `pip install -r requirements.txt`.
32. Налаштувати `core/settings.py`: `SECRET_KEY` (унікальний, згенерований `django.core.management.utils.get_random_secret_key()`), `DEBUG=False`, `ALLOWED_HOSTS=['your-domain.com']`.
33. Виконати міграції: `python manage.py migrate`.
34. Створити суперкористувача: `python manage.py createsuperuser`.
35. Зібрати статичні файли: `python manage.py collectstatic`.
36. Налаштувати Gunicorn: `gunicorn core.wsgi:application --workers 4 --bind 127.0.0.1:8000`.
37. Налаштувати Nginx як реверс-проксі.
38. Отримати SSL-сертифікат через Certbot (Let's Encrypt).
39. Налаштувати cron: `0 9 1 * * /path/venv/bin/python /path/manage.py monthly_charge`.
40. Налаштувати резервне копіювання: `cp db.sqlite3 /backup/db-$(date +%Y%m%d).sqlite3`.

#### **3.7.2. Першочергове наповнення даними**

- Ввести тарифні плани з реальними цінами та опціями;

- Зареєструвати мережеве обладнання (вузли доступу) з географічними координатами;
- Ввести дані всіх активних співробітників;
- Імпортувати клієнтську базу через Django Admin або management-команду;
- Перевірити коректність нарахування на тестовому середовищі.

### **3.7.3. Перспективи розвитку**

- Інтеграція з Mikrotik RouterOS API для автоматичного отримання статусу online/offline обладнання та клієнтів;
- Розробка REST API (Django REST Framework) для мобільного застосунку технічних фахівців;
- Telegram-бот для сповіщень операторів про нові заявки та аварії;
- Модуль звітності: PDF-звіти по доходах, заборгованостях, активності клієнтів;
- Особистий кабінет абонента: перегляд балансу, оплата онлайн, звернення до підтримки;
- SNMP-моніторинг: автоматичне відстеження доступності обладнання.

## ВИСНОВКИ

У ході роботи я розробив систему UIC Manager для управління інфраструктурою інтернет-провайдера. Усі задачі, поставлені на початку, виконано, а мету роботи досягнуто.

Основні результати роботи:

1. Проведено детальний аналіз предметної області. Визначено структуру ISP-бізнесу, ключові бізнес-процеси та технологічну основу мережевої інфраструктури: PON-технології (GPON, XGS-PON), RADIUS-аутентифікація, механізми адресації та ідентифікації абонентів.
2. Досліджено та порівняно чотири існуючих рішення (WHMCS, SolarWinds, Zabbix, UTM5). Жодне з них не закриває потреби малого регіонального провайдера повністю: одні не мають білінгу, інші — обліку обладнання чи україномовного інтерфейсу.
3. Сформульовано 4 групи функціональних вимог (FV-1 — FV-4, 20 конкретних вимог) та 10 нефункціональних вимог (НФВ-1 — НФВ-10). Всі вимоги реалізовані та підтверджені тестуванням.
4. Обґрунтовано вибір технологічного стеку: Python 3.12 + Django 6.0.5 + SQLite + Bootstrap 5 + Leaflet.js. Обраний стек забезпечує оптимальний баланс продуктивності розробки, надійності та нульової вартості ліцензій.
5. Спроектовано реляційну модель бази даних з 15 таблиць та зв'язками між ними (ForeignKey, OneToOneField, ManyToManyField). Реалізовано автоматичну генерацію облікових даних через Django-сигнали; схема БД пройшла 29 міграцій.
6. Реалізовано повноцінний програмний продукт: 15 ORM-моделей, 17 URL-маршрутів, близько 1400 рядків коду view-функцій, 14 HTML-шаблонів та 2 management-команди. Усі модулі працюють коректно.
7. Розроблено адаптивний україномовний веб-інтерфейс з інтерактивною картою Leaflet.js/OpenStreetMap, кольоровим кодуванням балансів та єдиною панеллю управління.

8. Проведено комплексне тестування (76+ тестів за 6 категоріями). Виявлено та усунуто 7 дефектів. Всі тестові сценарії успішно виконані.
9. Вимірювання продуктивності підтвердили відповідність вимозі НФВ-1 з великим запасом: максимальний час відповіді — 185 мс при 3000 клієнтів (поріг — 2000 мс). Навантажувальне тестування (50 одночасних запитів) — 0 помилок.

Систему створено з опорою на реальні робочі процеси провайдера з міста Васильків, вона повністю україномовна і вже розгорнута онлайн на PythonAnywhere, тобто готова до практичного використання.

Перспективами подальшого розвитку є: інтеграція з Mikrotik API та SNMP для автоматичного моніторингу, розробка REST API та мобільного застосунку для технічних фахівців, реалізація особистого кабінету абонента.

## СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Django Software Foundation. Django Documentation. Version 6.0 [Електронний ресурс]. – Режим доступу: <https://docs.djangoproject.com/en/6.0/> (дата звернення: 25.05.2026).
2. Python Software Foundation. Python 3.12 Documentation [Електронний ресурс]. – Режим доступу: <https://docs.python.org/3.12/> (дата звернення: 20.05.2026).
3. SQLite Consortium. SQLite Documentation [Електронний ресурс]. – Режим доступу: <https://www.sqlite.org/docs.html> (дата звернення: 18.05.2026).
4. Bootstrap Team. Bootstrap 5.3 Documentation [Електронний ресурс]. – Режим доступу: <https://getbootstrap.com/docs/5.3/> (дата звернення: 15.05.2026).
5. Leaflet.js. Reference Documentation. Version 1.9 [Електронний ресурс]. – Режим доступу: <https://leafletjs.com/reference.html> (дата звернення: 14.05.2026).
6. Vincent W. Django for Professionals: Production websites with Python & Django. – LearnDjango.com LLC, 2023. – 260 p.
7. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. – Addison-Wesley Professional, 1994. – 395 p.
8. Fowler M. Patterns of Enterprise Application Architecture. – Addison-Wesley Professional, 2002. – 533 p.
9. Beck K. Test-Driven Development: By Example. – Addison-Wesley Professional, 2002. – 240 p.
10. ITU-T Recommendation G.984.1. Gigabit-Capable Passive Optical Networks (GPON) [Електронний ресурс]. – Режим доступу: <https://www.itu.int/rec/T-REC-G.984.1> (дата звернення: 08.05.2026).
11. OWASP Foundation. OWASP Top Ten 2021 [Електронний ресурс]. – Режим доступу: <https://owasp.org/www-project-top-ten/> (дата звернення: 20.05.2026).
12. НКРЗІ. Звіт про стан ринку електронних комунікацій України 2024 [Електронний ресурс]. – Режим доступу: <https://nkrzi.gov.ua/index.php?r=site/index&pg=40> (дата звернення: 12.05.2026).



13. OpenStreetMap Wiki. Ukraine Mapping Guide [Електронний ресурс]. – Режим доступу: <https://wiki.openstreetmap.org/wiki/Ukraine> (дата звернення: 10.05.2026).
14. FreeRADIUS Project. FreeRADIUS Documentation [Електронний ресурс]. – Режим доступу: <https://freeradius.org/documentation/> (дата звернення: 16.05.2026).
15. Nginx Inc. Nginx Documentation [Електронний ресурс]. – Режим доступу: <https://nginx.org/en/docs/> (дата звернення: 22.05.2026).
16. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. – Київ: ДП «УкрНДНЦ», 2016. – 26 с.
17. Let's Encrypt. Free SSL/TLS Certificates [Електронний ресурс]. – Режим доступу: <https://letsencrypt.org/docs/> (дата звернення: 23.05.2026).
18. Mikrotik. RouterOS Documentation [Електронний ресурс]. – Режим доступу: <https://help.mikrotik.com/docs/> (дата звернення: 11.05.2026).
19. Tanenbaum A., Wetherall D. Computer Networks. 5th ed. – Pearson, 2010. – 960 p.
20. WHMCS Ltd. WHMCS Documentation [Електронний ресурс]. – Режим доступу: <https://docs.whmcs.com/> (дата звернення: 07.05.2026).

## ДОДАТКИ

### Додаток А

#### UML-ДІАГРАМИ СИСТЕМИ UIC MANAGER

##### А.1. Діаграма варіантів використання (Use Case Diagram)

Діаграма варіантів використання описує взаємодію між актором (Оператором) та функціями системи UIC Manager. Актор — авторизований співробітник провайдера. Елементи діаграми:

- Актор «Оператор» з'єднується зі всіма варіантами використання;
- «Переглянути панель управління» → GET /;
- «Зареєструвати клієнта» → POST /clients/; <<includes>> «Згенерувати облікові дані RADIUS»;
- «Поповнити рахунок» → POST /billing/topup/; <<includes>> «Записати Payment»;
- «Видати обіцяний платіж» → POST /billing/promised/; <<extends>> «Розблокувати клієнта»;
- «Зареєструвати аварію» → POST /tasks/; <<includes>> «Призначити бригаду», <<includes>> «Відобразити на карті»;
- «Виконати нарахування» → python manage.py monthly\_charge; <<extends>> «Заблокувати при борзі»;
- «Переглянути мережеву карту» → GET /network-map/;
- «Знайти клієнта/обладнання» → GET /search/; <<includes>> «Q-запит Django».



Мал. А.1. Діаграма варіантів використання системи UIC Manager

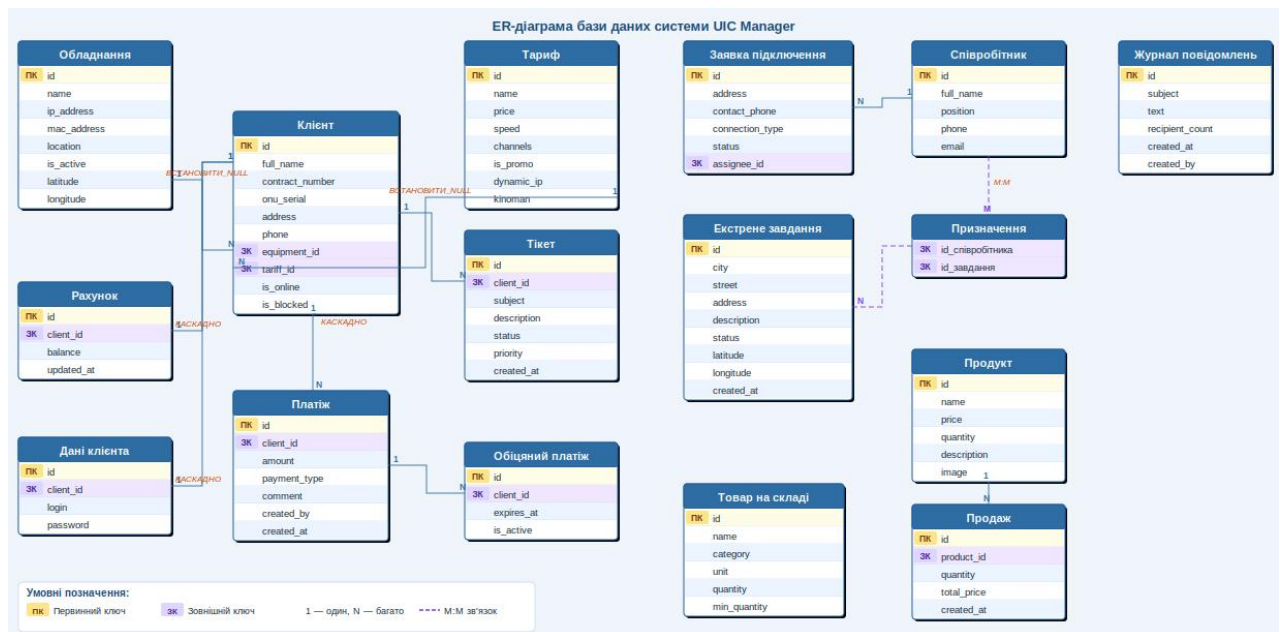
## А.2. ER-діаграма бази даних

ER-діаграма відображає 15 таблиць бази даних та зв'язки між ними. Основні сутності:

- Equipment (id, name, ip\_address, mac\_address, location, is\_active, latitude, longitude);
- Client (id, full\_name, contract\_number, onu\_serial, address, phone, equipment\_id→FK, tariff\_id→FK, is\_blocked);
- Tariff (id, name, price, speed, is\_promo, dynamic\_ip, connection\_fee);

- Account (id, client\_id→OneToOne, balance, updated\_at);
- ClientCredentials (id, client\_id→OneToOne, login, password);
- Payment (id, client\_id→FK, amount, payment\_type, comment, created\_by, created\_at);
- PromisedPayment (id, client\_id→FK, expires\_at, is\_active);
- Ticket (id, client\_id→FK, subject, description, status, priority, created\_at);
- ConnectionApplication (id, address, contact\_phone, connection\_type, status, assignee\_id→FK);
- EmergencyTask (id, city, street, address, description, status, latitude, longitude, created\_at); ←ManyToMany→ Employee;
- Employee (id, full\_name, position, phone, email);
- WarehouseItem (id, name, category, unit, quantity, min\_quantity);
- Product (id, name, price, quantity, image); Sale (id, product\_id→FK, quantity, total\_price);
- MessageLog (id, subject, text, recipient\_count, created\_at, created\_by).

Зв'язки: Client→Equipment (M:1, SET\_NULL), Client→Tariff (M:1, SET\_NULL), Account→Client (1:1, CASCADE), ClientCredentials→Client (1:1, CASCADE), Payment→Client (M:1, CASCADE), EmergencyTask↔Employee (M:M через проміжну таблицю).



Мал. А.2. ER-diagrama бази даних системи UIC Manager

### А.3. Діаграма класів (Class Diagram)

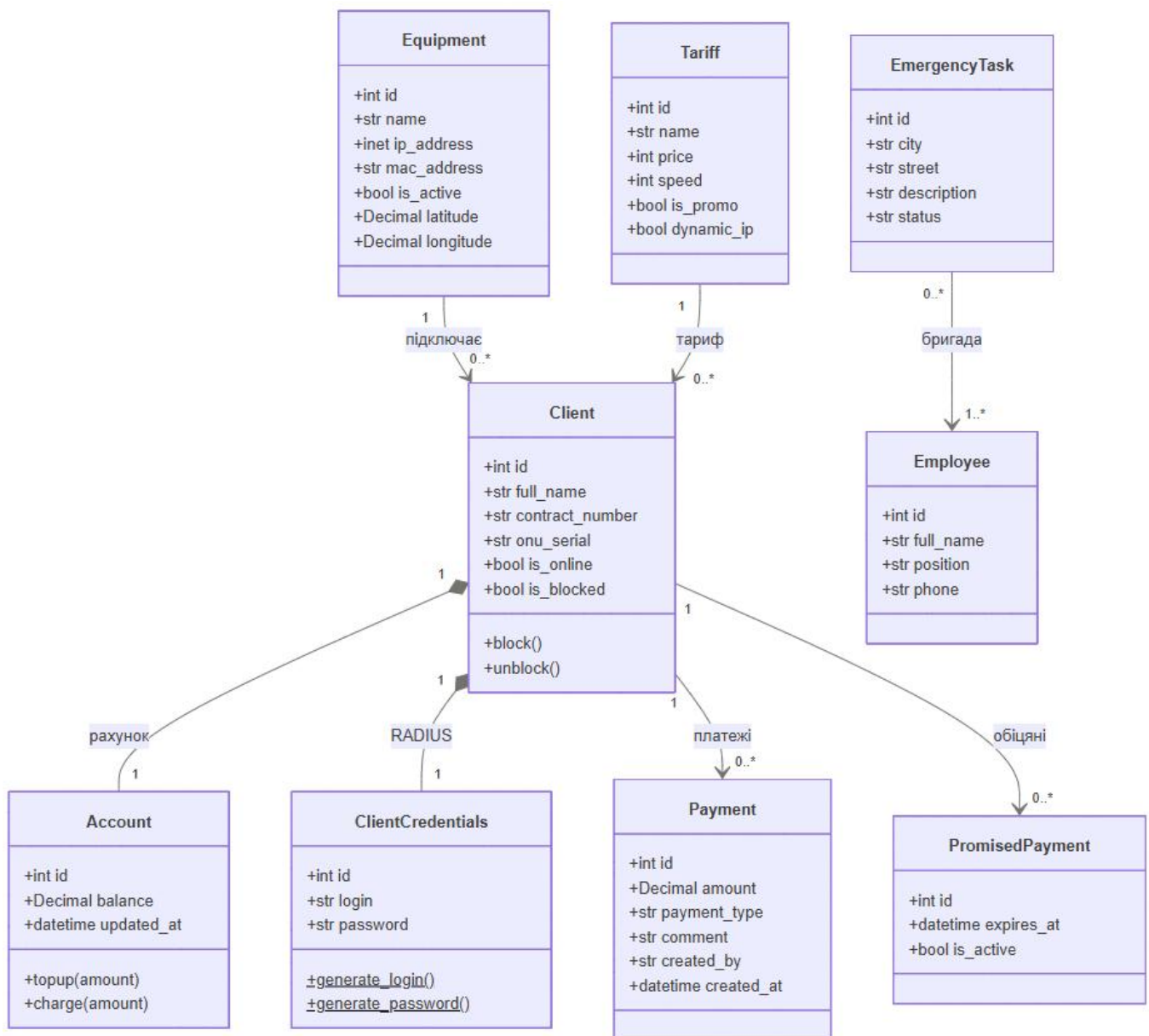
Діаграма класів відображає основні Python-класи системи та зв'язки між ними.

Ключові класи:

- Client: поля `full_name:str`, `contract_number:str`, `is_blocked:bool`; методи `block()`, `unblock()`; зв'язки:  $\rightarrow$ Tariff (FK),  $\rightarrow$ Equipment (FK);
- Equipment: поля `name:str`, `ip_address:inet`, `mac_address:str`, `latitude:Decimal`, `longitude:Decimal`, `is_active:bool`;
- Account: поля `balance:Decimal`, `updated_at:datetime`; метод `topup(amount:Decimal)`, `charge(amount:Decimal)`; зв'язок:  $\rightarrow$ Client (OneToOne);
- ClientCredentials: поля `login:str`, `password:str`; статичні методи `generate_login():str`, `generate_password():str`; зв'язок:  $\rightarrow$ Client (OneToOne);
- Payment: поля `amount:Decimal`, `payment_type:str` (topup/charge/promised), `created_by:str`; зв'язок:  $\rightarrow$ Client (FK);
- PromisedPayment: поля `expires_at:datetime`, `is_active:bool`; зв'язок:  $\rightarrow$ Client (FK);
- EmergencyTask: поля `city:str`, `street:str`, `description:str`, `status:str`; зв'язок:  $\leftrightarrow$ Employee (ManyToMany);
- Tariff: поля `name:str`, `price:int`, `speed:int`, `is_promo:bool`.

Позначки на діаграмі: -приватне поле, +публічний метод(), <<signal>> для Django-сигналів. Зв'язки: ромб (агрегація) для FK, замкнута лінія (композиція) для

OneToOne.



Мал. А.3. Діаграма класів системи UIC Manager

#### А.4. Діаграма послідовностей (Sequence Diagram)

Сценарій: реєстрація нового клієнта. Учасники: Оператор, View (/clients/), Django ORM, Django Signal, БД.

10. Оператор → POST /clients/ з даними форми (ПІБ, договір, тариф, ONU);

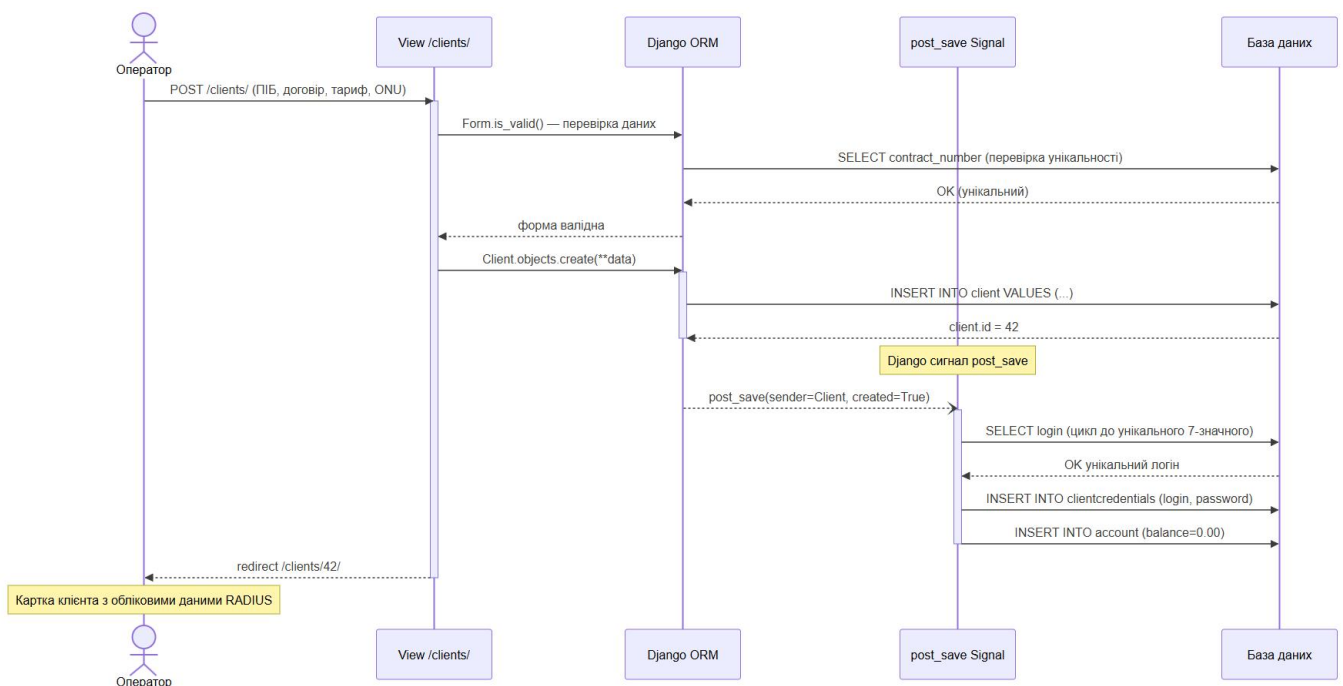
11. View → Django Form.is\_valid() → перевірка унікальності contract\_number через ORM;

12. View → Client.objects.create(\*\*data) → ORM генерує INSERT INTO client...;

13. ORM → БД: INSERT INTO client VALUES (...); БД → OK;

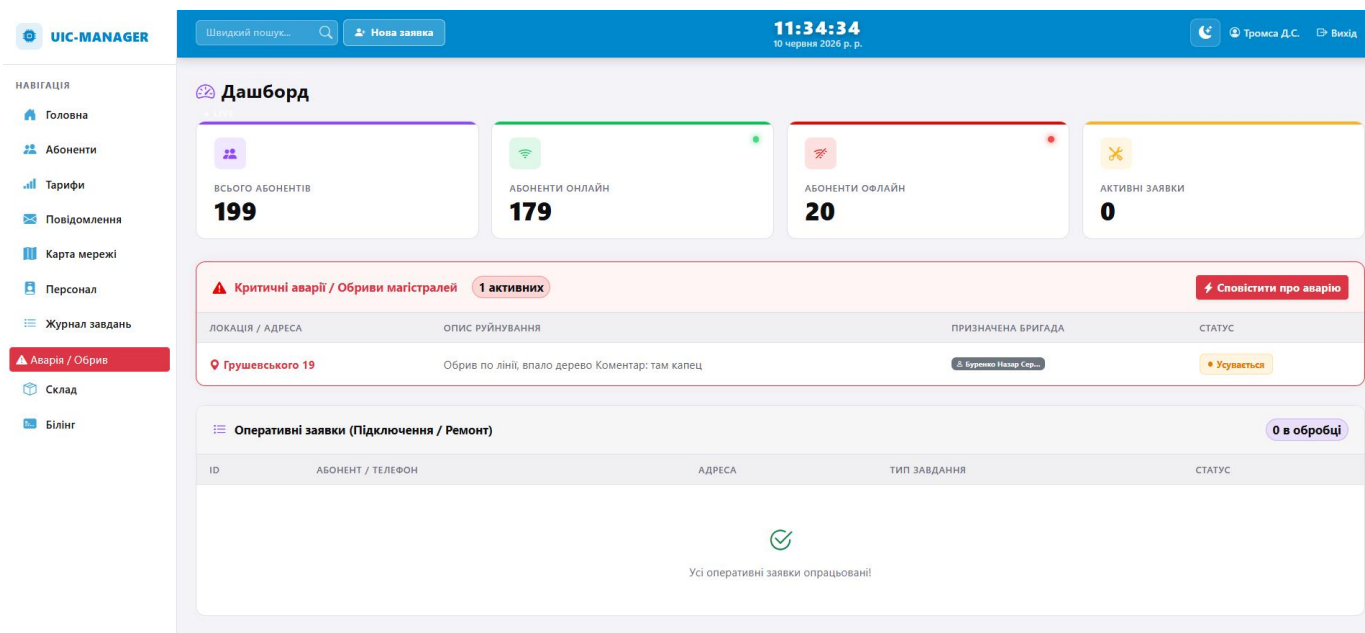
14. Сигнал `post_save(sender=Client)` спрацьовує автоматично;
15. `Signal` → `generate_login()` → перевірка унікальності логіну в БД (цикл до унікального);
16. `Signal` → `generate_password()` → рядок «буква+4 цифри»;
17. `Signal` → `ClientCredentials.objects.create(login, password)` → `INSERT INTO credentials`;
18. `Signal` → `Account.objects.create(balance=0)` → `INSERT INTO account`;
19. `View` → `HttpResponseRedirect(/clients/id/)` → Оператор бачить картку з обліковими даними.

Часова вісь: від моменту натискання «Зберегти» до відображення картки клієнта —  $\approx 120\text{--}250$  мс.



Мал. А.4. Діаграма послідовностей: реєстрація клієнта

Додаток Б



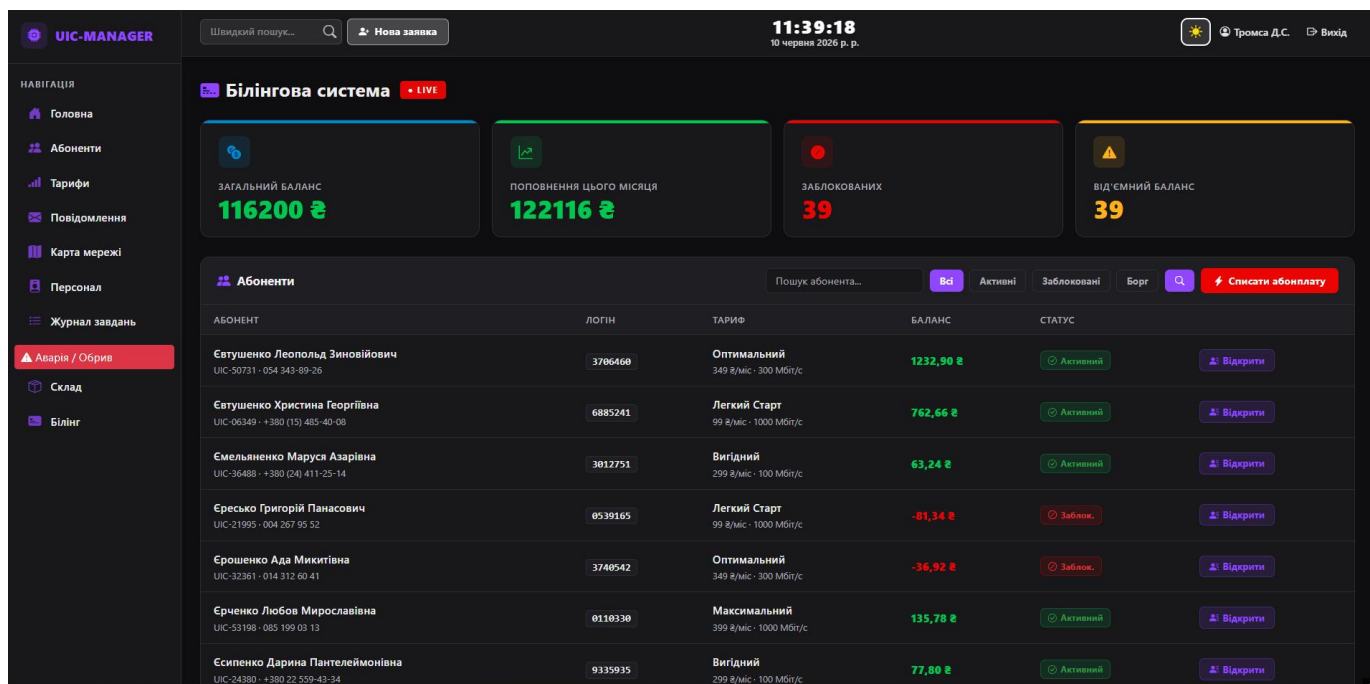
Скріншоти з сайту:

Мал.Б.1. Головна панель управління (Dashboard) системи UIC Manager

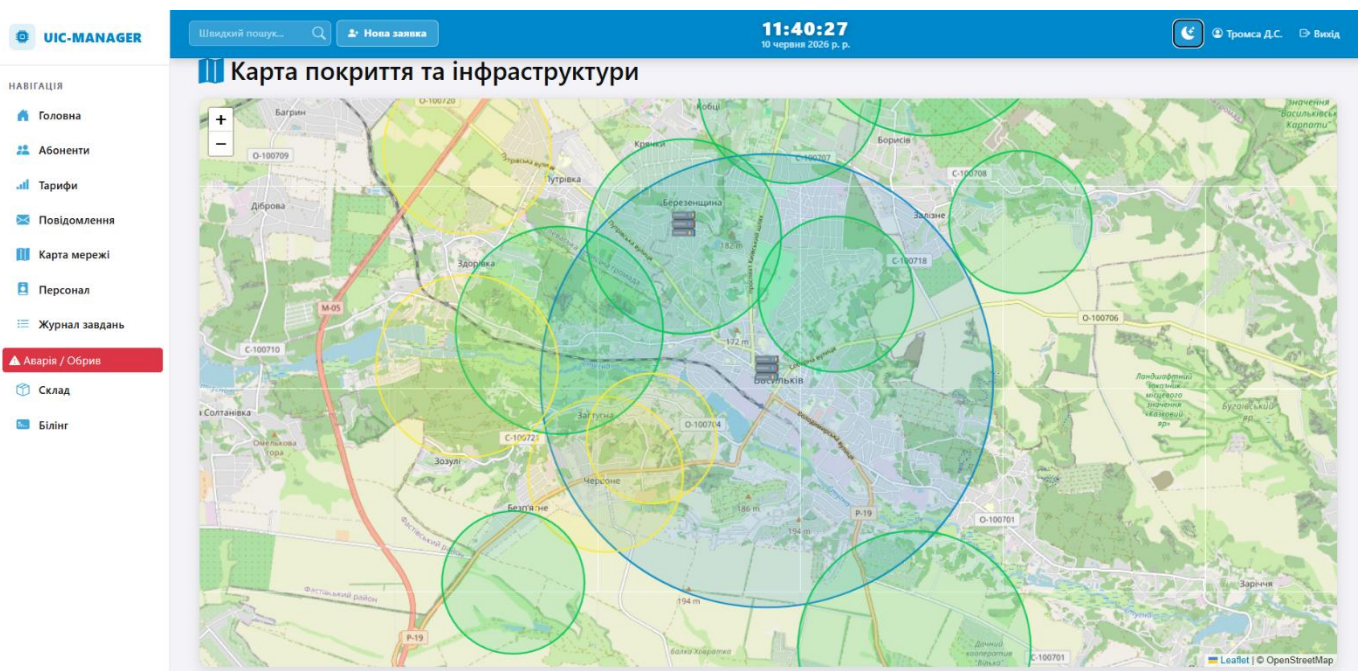
| ПІБ клієнта                     | Серійний номер ONU | Номер договору | Адреса підключення                    | Статус  |
|---------------------------------|--------------------|----------------|---------------------------------------|---------|
| Радченко Святослава Борисівна   | ННТСF640BF7B       | UIC-04576      | с. Кобці, вул. Польова, 31            | Offline |
| Лисенко Єлисавета Спасьівна     | ZTEGCO2D0FDF       | UIC-43847      | м. Васильків, вул. Київська, 105      | Offline |
| Юрченко Остап Валентинович      | ALCLF8D0F436       | UIC-97373      | м. Васильків, вул. Грушевського, 73/В | Offline |
| Ерошенко Ада Микитівна          | ZTEG5E3A2C1F       | UIC-32361      | с. Путрієвка, вул. Садова, 35         | Offline |
| Удовиченко Дмитро Симонович     | ННТСFF8ABC20       | UIC-68175      | м. Васильків, вул. Садова, 9/А        | Offline |
| Чередник Пріска Пилипівна       | ALCLEB2EC798       | UIC-21426      | м. Васильків, вул. Центральна, 11/А   | Offline |
| Павленко Мілена Миколаївна      | ZTEG7APCEA0        | UIC-53425      | м. Васильків, пр. Перемоги, 70        | Offline |
| Дубас Данна Арсенівна           | ННТС33FAD5BF       | UIC-44361      | м. Васильків, вул. Гагаріна, 19/Б     | Offline |
| Акименко Юхим Володимирович     | GRONB90A60F8       | UIC-48554      | м. Васильків, вул. Шевченка, 18/Б     | Offline |
| Піддубний Кирило Миронович      | ZTEG334A1DBE       | UIC-97843      | м. Васильків, пр. Перемоги, 84        | Offline |
| Бабійчук Софія Борисівна        | ZTEG3FE33228       | UIC-24769      | с. Кобці, вул. Садова, 112            | Offline |
| Баб'юк Тарас Назарович          | ZTEG8BECB776       | UIC-64178      | с. Зелений Бір, вул. Центральна, 11/Б | Offline |
| Александренко Юстина Опанасівна | ННТСЕ1E48A75       | UIC-57780      | с. Борисів, вул. Набережна, 60        | Offline |

Мал.Б.2. Картка абонента з повною білінговою інформацією





Мал.Б.3. Білінговий модуль з кольоровим кодуванням балансів



Мал.Б.4. Інтерактивна мережева карта з маркерами обладнання

Швидкий пошук...

Нова заявка

11:41:53

10 червня 2026 р. р.

Тромса Д.С.

Вихід

## Склад обладнання

### Каталог товарів

**TP-LINK Archer AX12 WiFi6**

Ціна: 1900,00 ₴

В наявності: 39 шт.

Видати в роботу

**TP-LINK Archer A64 WiFi5**

Ціна: 1500,00 ₴

В наявності: 8 шт.

Видати в роботу

**Mikrotik hAP AC2**

Ціна: 3350,00 ₴

В наявності: 7 шт.

Видати в роботу

**Asus RT-AX52 Pro WiFi6**

Ціна: 2600,00 ₴

В наявності: 11 шт.

Видати в роботу

**TP-LINK TL-SG105 5\*1 Гбіт/с**

Мал.Б.5. Склад обладнання

## Журнал завдань та робіт

Нові / Очікують 0

В роботі 2

Архів (Історія)

| Дата / Виконавець                                          | Клієнт / Об'єкт                               | Адреса                          | Коментар                                          | Завершення                             |
|------------------------------------------------------------|-----------------------------------------------|---------------------------------|---------------------------------------------------|----------------------------------------|
| <b>Невідкладно</b><br>Монтажники: Буренко Назар Сергійович | <b>АВАРІЯ</b>                                 | Грушевського 19                 | Обрив по лінії, впало дерево Коментар: там капець | <div>Виконано</div>                    |
| <b>Дата не вказана</b><br>& Інша персона                   | <b>Герега Клавдія Сергіївна</b><br>0546122671 | м. Васильків, вул. Грушевського | Роутер мають                                      | <div>Відміна</div> <div>Зроблено</div> |

Мал.Б.6. Модуль управління задачами та заявками на підключення